

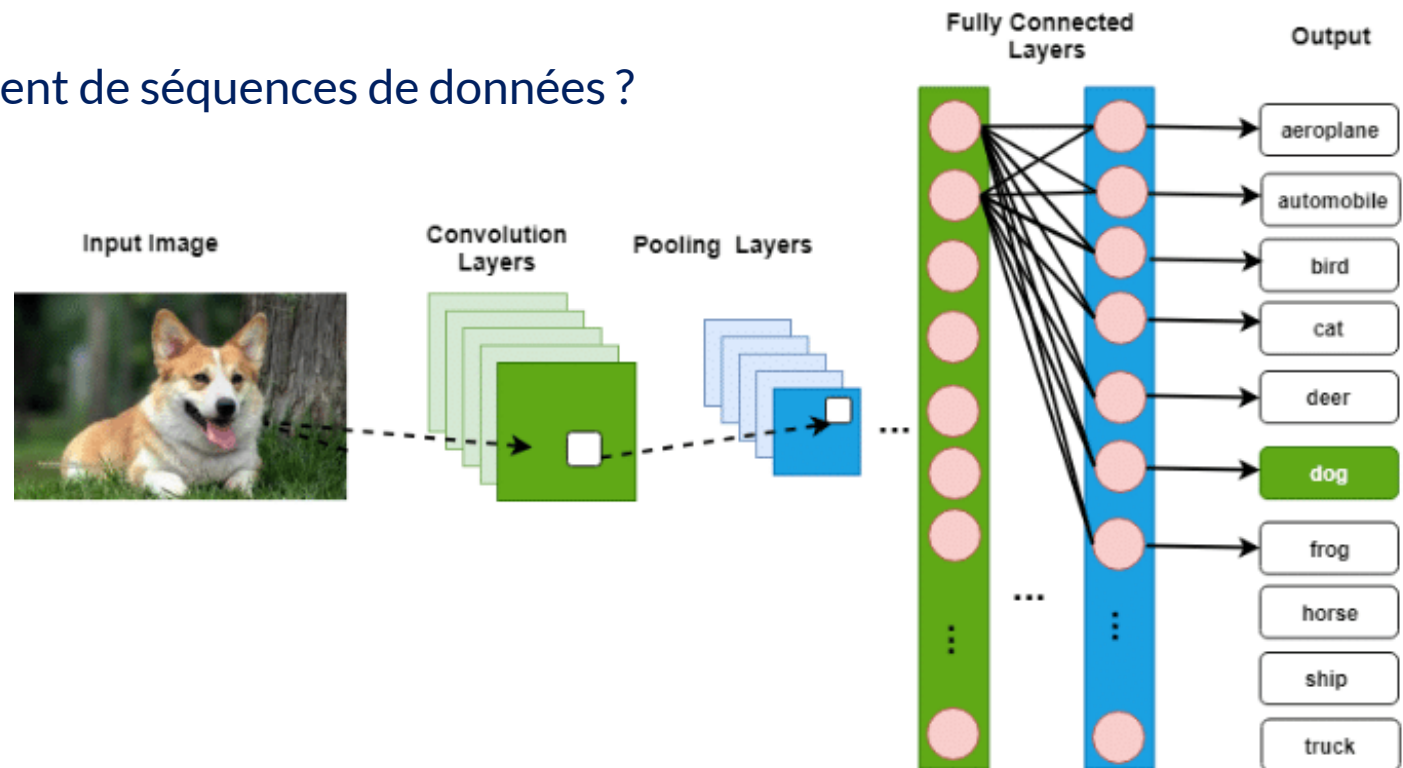
Méthodes Numériques et Modélisation

Fondements de l'intelligence artificielle et applications aux sciences atmosphériques



CE QU'ON SAIT DES RÉSEAUX DE NEURONES

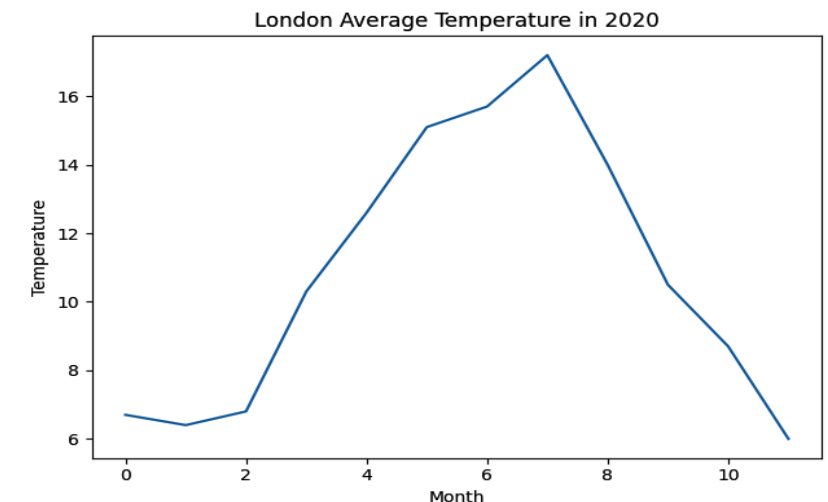
- Jusqu'à présent nous avons utilisé les réseaux de neurones pour des régressions et des classifications, surtout dans le cas des images
 - Full (Dense) Neural Networks
 - Convolutional Neural Networks
- Serait-il possible de les utiliser pour le traitement de séquences de données ?



LES SÉRIES TEMPORELLES

- Les séries temporelles sont des séquences d'observation avec une dépendance **chronologique**
- Les données peuvent être facilement organisées sous la forme d'un tableau
 - Souvent, la visualisation aide à comprendre les dépendances
- On peut considérer deux domaines d'étude :
 - Analyse des séries temporelles
 - Identification des structures et éléments clé
 - Tendances, cycles, corrélations
 - Inclut la préparation des données, la visualisation
 - Prédiction de séries temporelles
 - Développement de modèles et obtention de prévisions

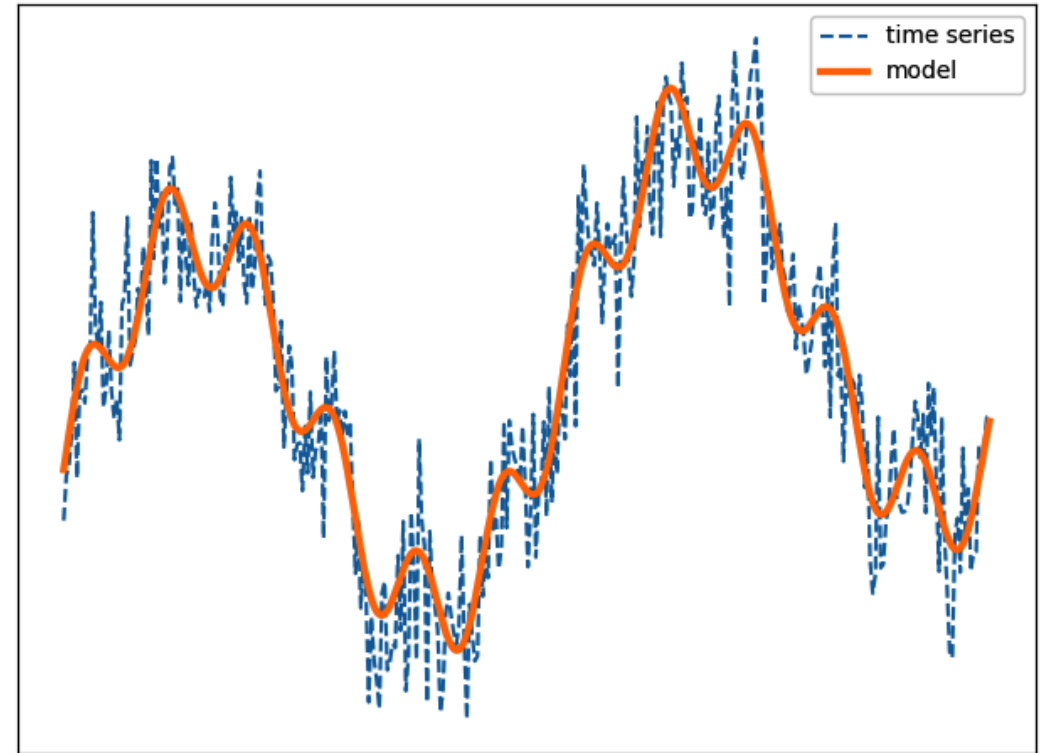
Mois	Température Moyenne (Celsius)
Janvier	6,7
Février	6,4
Mars	6,8
Avril	10,3
Mai	12,6
Juin	15,1
Juillet	15,7
Août	17,2
Septembre	14
Octobre	10,5
Novembre	8,7
Décembre	6



QUELS TYPES DE PROBLÈME

■ Modélisation

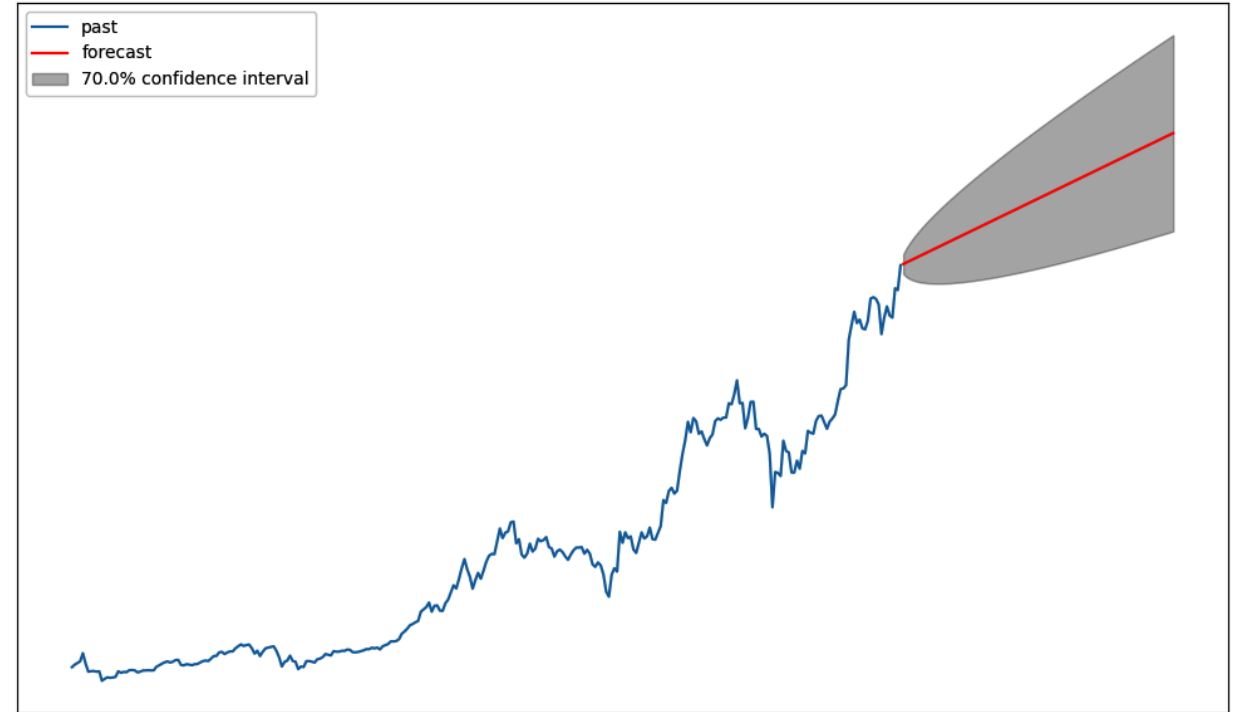
- La modélisation implique la construction d'un modèle qui décrit le processus observé dans la série temporelle
- Son objectif est de comprendre la nature et la logique du processus
 - La présence d'exoplanètes en raison de la variation de la luminosité/trajectoire d'une étoile
 - La structure du cœur à partir d'un électrocardiogramme
 - Les préférences des consommateurs



QUELS TYPES DE PROBLÈME

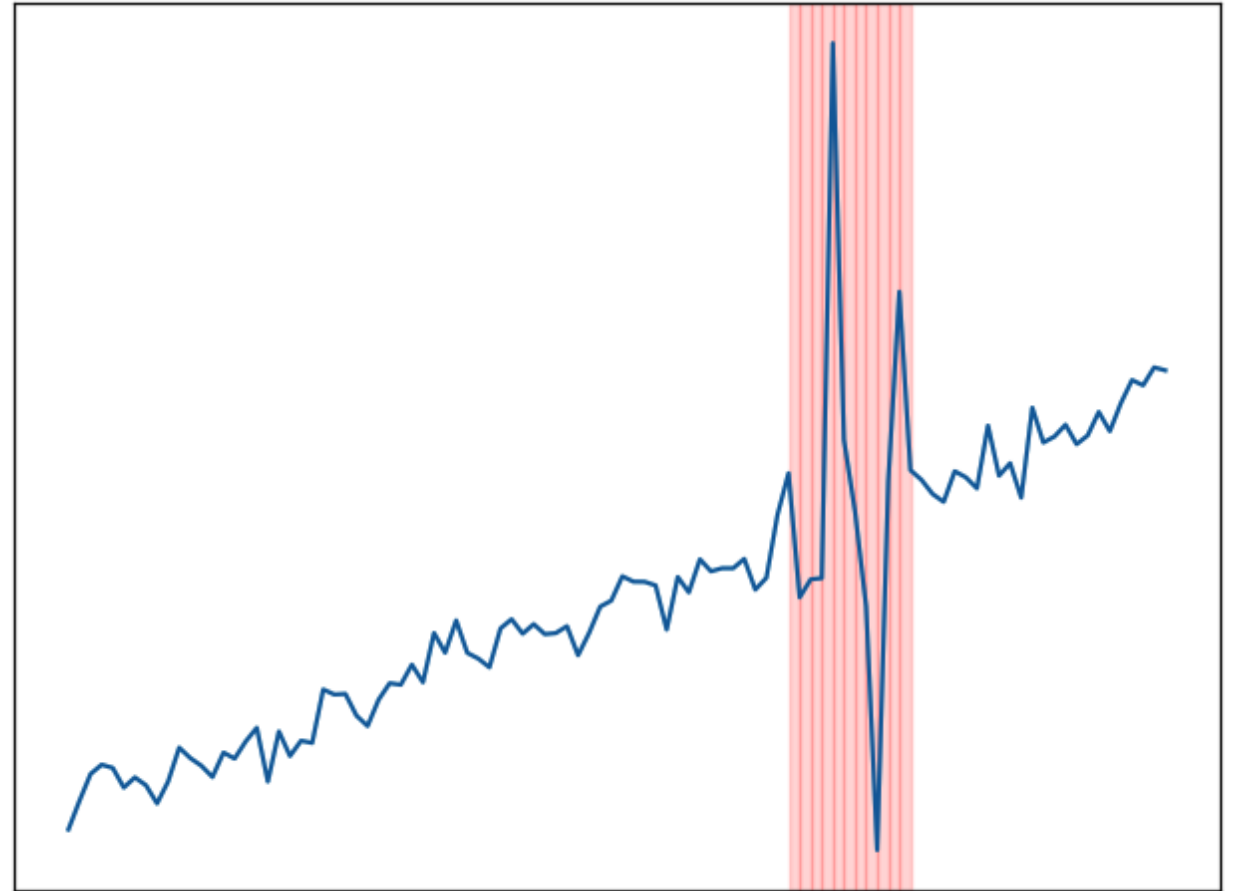
■ Forecast

- Etudier le comportement d'une série temporelle permet de trouver la valeur la plus probable dans une étape future
- Une prédiction n'est jamais une solution exacte, juste une suggestion dans une **fourchette d'erreur**
- Toute série temporelle a une composante aléatoire, il est impossible de prédire le futur avec 100% de probabilité



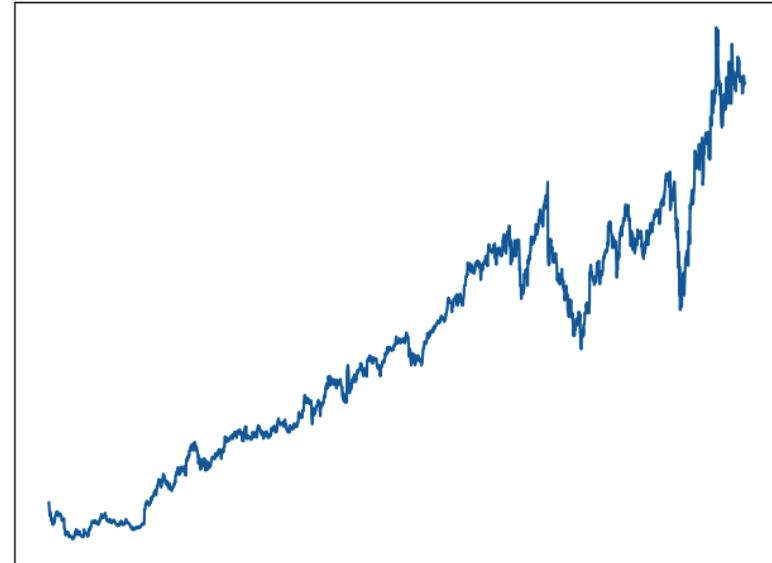
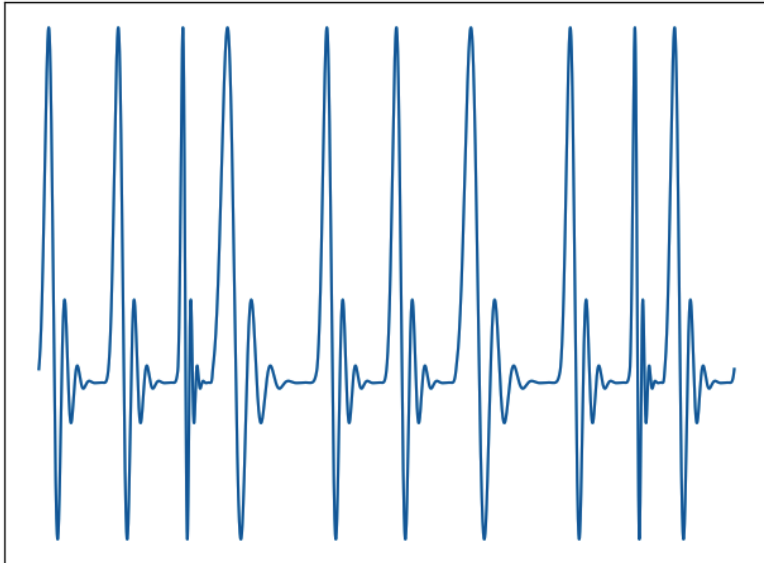
QUELS TYPES DE PROBLÈME

- **Détection d'anomalies**
- Une dernière utilisation des modèles est l'identification de situations hors-norme
 - Changements inattendus de température des équipements
 - Détection de fraudes (activités bancaires atypiques)
 - Activité cardiaque irrégulière
- Le "fourchettes" d'erreur permettent de définir les marges de tolérance



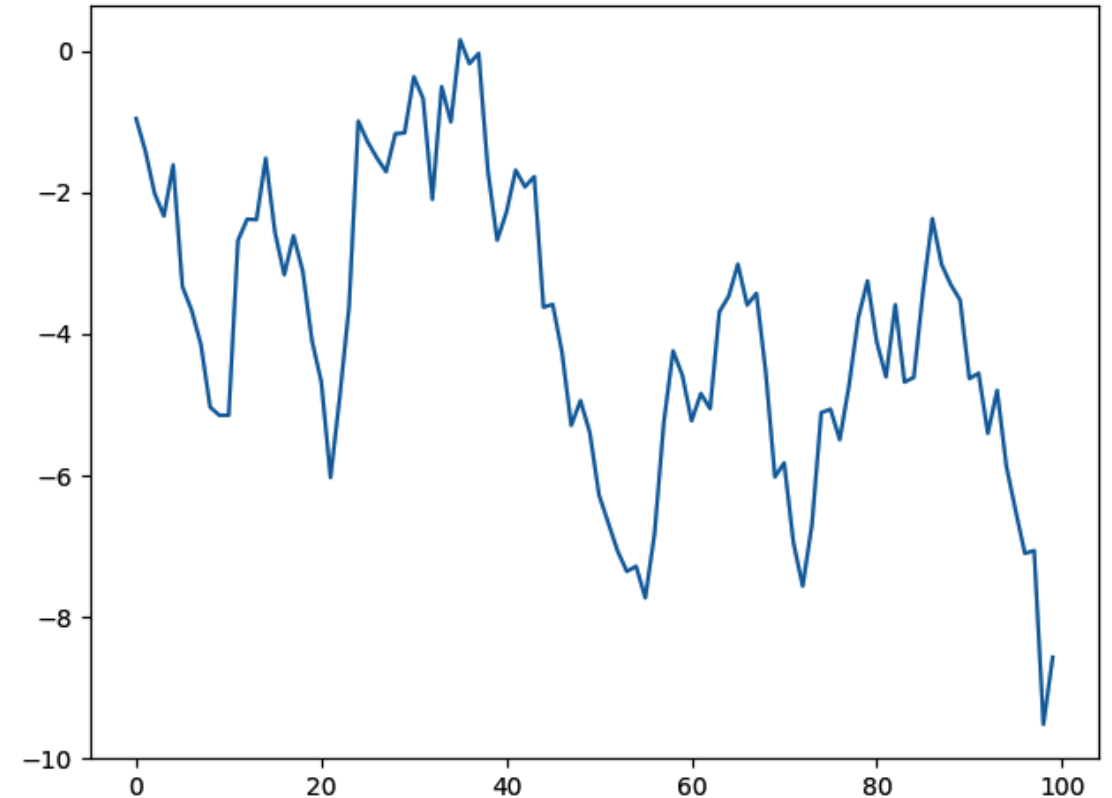
CARACTÉRISATION DES SÉRIES TEMPORELLES

- Les séries temporelles peuvent présenter différentes caractéristiques, mais on trouve souvent :
 - Des cycles
 - Des fluctuations (limitées)
 - Des tendances



CARACTÉRISATION DES SÉRIES TEMPORELLES

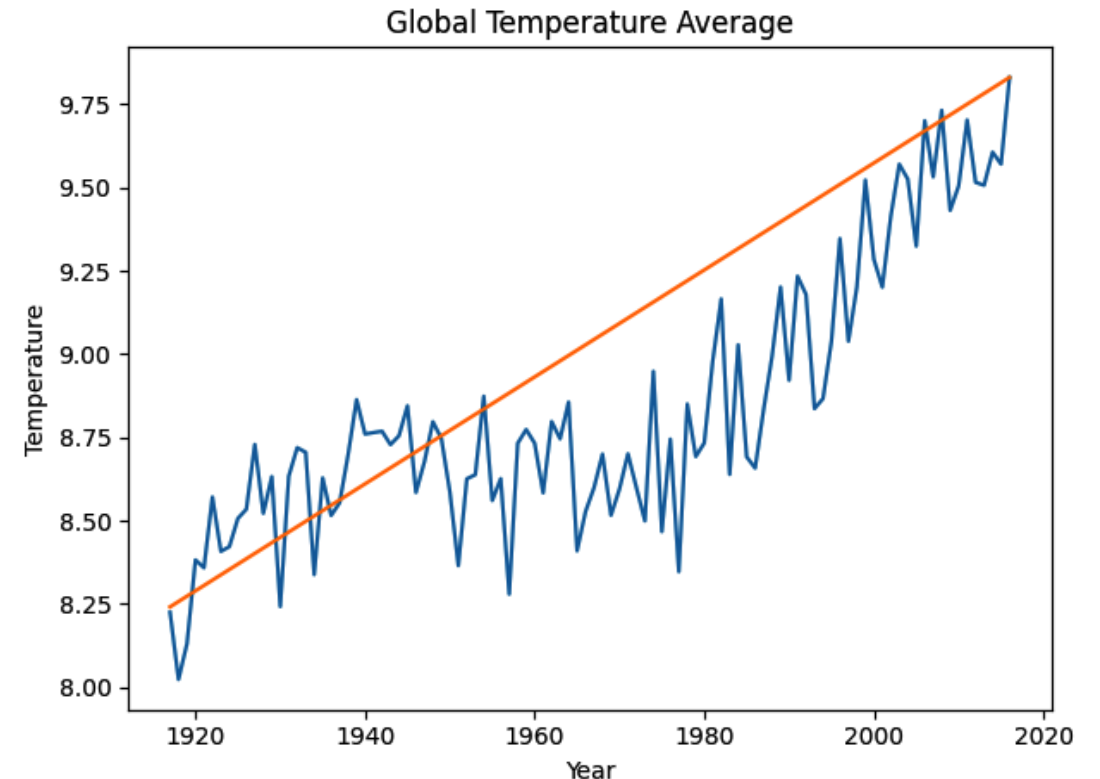
- **Random Walk**
 - C'est le comportement le plus simple observé
 - Caractérisé par une somme de variables aléatoires indépendantes suivant une distribution normale
 - $R_{t+1} = R_t + E_t$
- Un Random Walk est essentiellement un processus stochastique où l'état à un temps t est lié à l'état $t-1$



CARACTÉRISATION DES SÉRIES TEMPORELLES

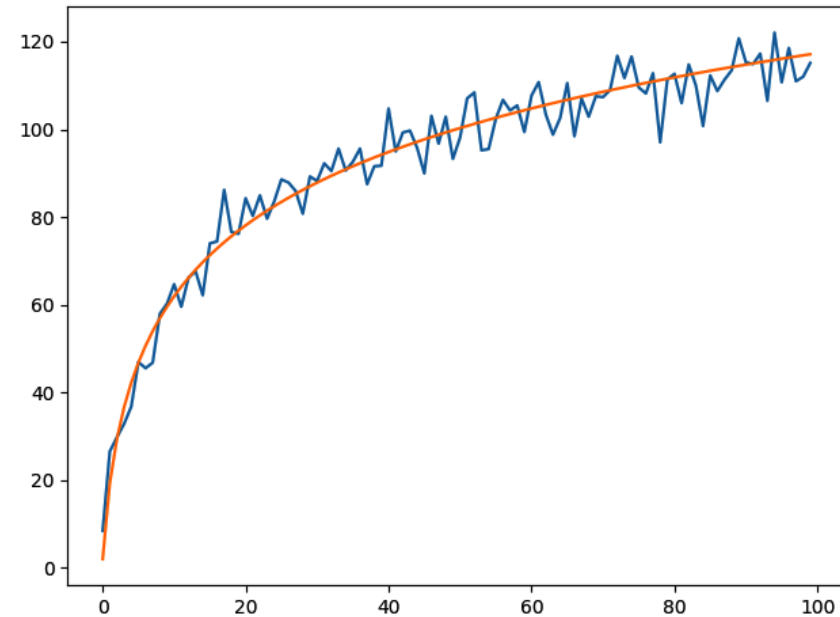
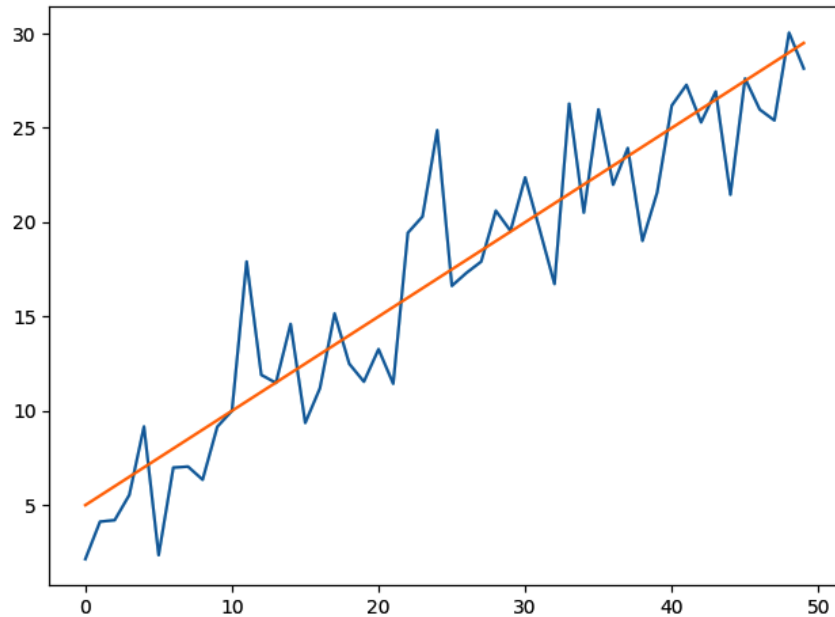
- **Trend (tendance)**

- Certaines séries temporelles observent une tendance (direction de progression)
- La modélisation doit prendre en compte la tendance car elle a un impact non-négligeable
 - Ex : s'entraîner juste avec des valeurs 1920-1980 donnerait probablement des prédictions sur une fourchette 8-9 degrés seulement
- $T_t = A + F_t + E_t$
 - Où F_t est une fonction (linéaire ou non)



CARACTÉRISATION DES SÉRIES TEMPORELLES

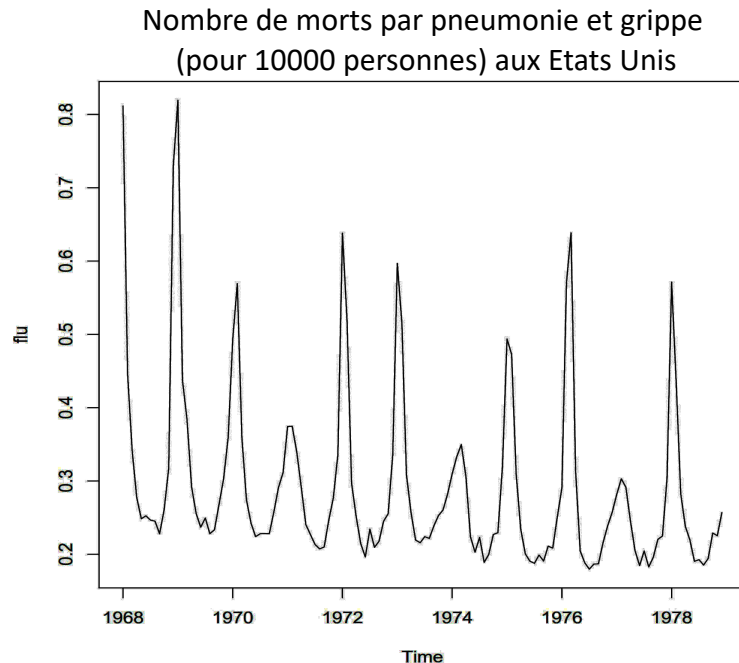
- Les tendances peuvent être linéaires ou non-linéaire
 - Exemple 1 : $A = 5$; $F = 0.5 * i$; $E = 3 * \text{random.gauss}(0, 1)$
 - Exemple 2 : $A = 2$; $F = 25 * \log(i)$; $E = 5 * \text{random.gauss}(0, 1)$



CARACTÉRISATION DES SÉRIES TEMPORELLES

■ Saisonnalité

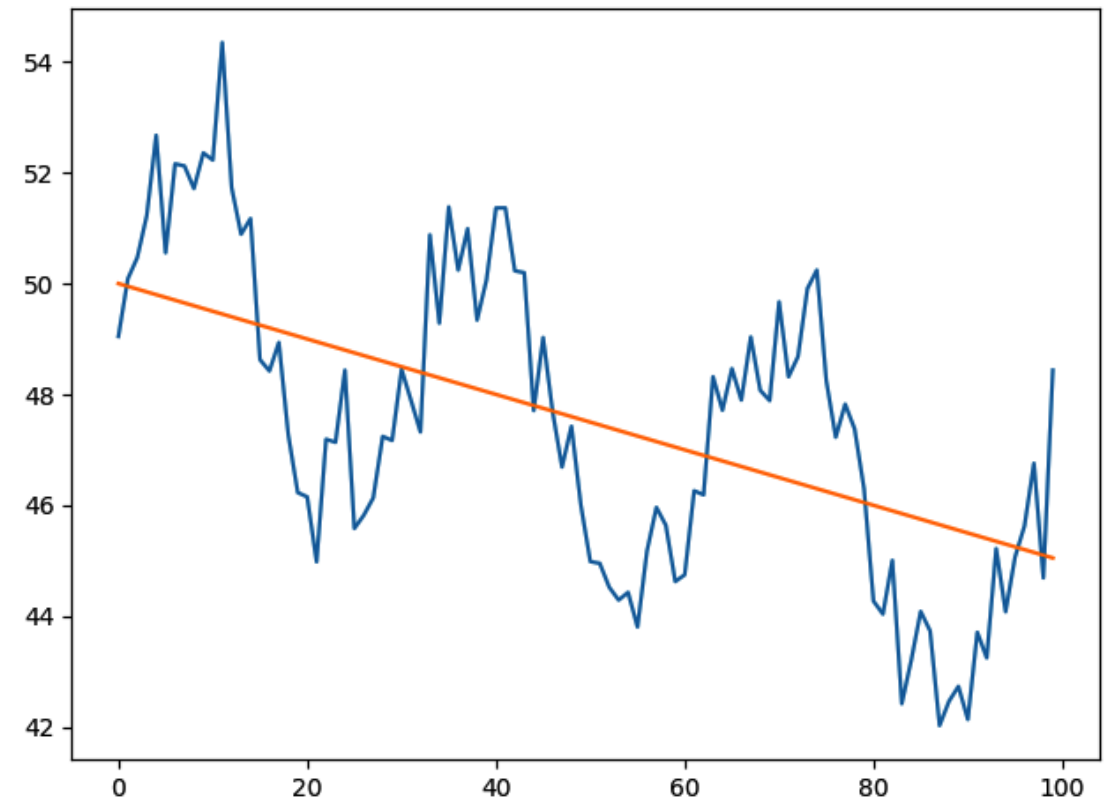
- C'est une variation répétitive dans la série temporelle
- Plusieurs processus sont cycliques par nature, comme par exemple le nombre de décès dus aux maladies respiratoires



- La série temporelle est donc impactée par une **fonction périodique**
 - $T_t = A + F_t + CS_t + E_t$
- Une fonction périodique est une fonction qui se répète à des intervalles réguliers
 - $S_{t+p} = S_t$

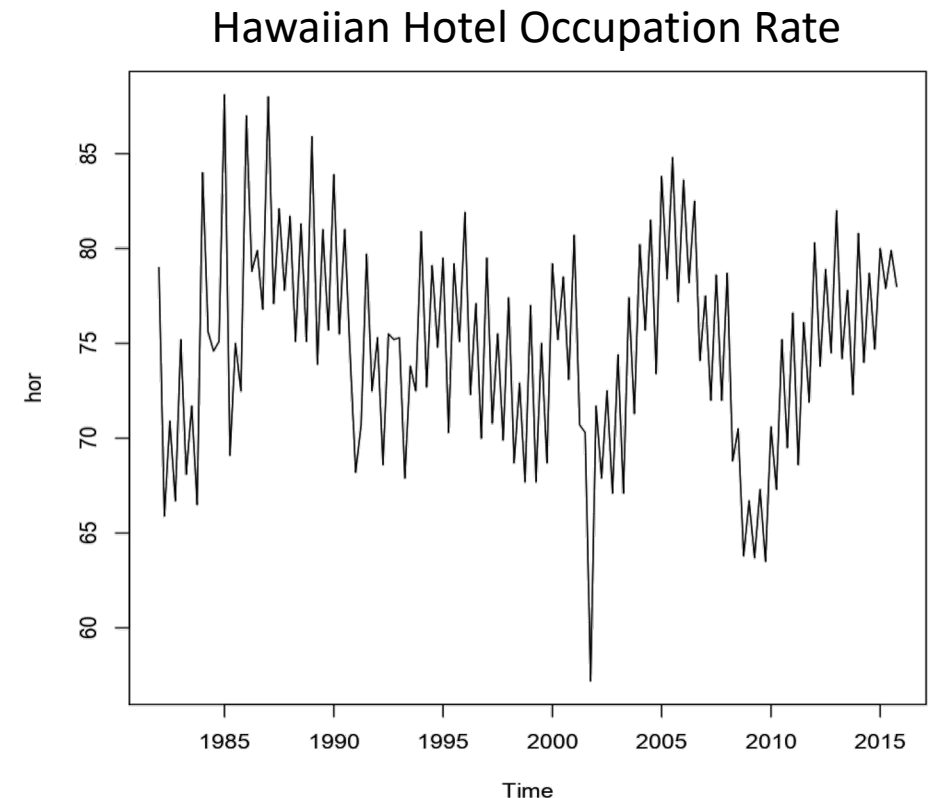
CARACTÉRISATION DES SÉRIES TEMPORELLES

- Les fonctions périodiques les plus courantes sont des fonctions trigonométriques
 - sinus, cosinus
- **Les fonctions périodiques peuvent se cumuler**
 - Des variations saisonnières (été – hiver)
 - Jour de la semaine (weekend ou pas)
 - Heure du jour
- Ex : $A = 50$; $B = -0.05$; $C = 1$; $S = 3$
- $Tendance = A + B \cdot i$
- $Saisonnalité = S \cdot \sin(i / 5)$
- $Bruit = C \cdot \text{random.gauss}(0, 1)$
- $T_i = Tendance[i] + Saisonnalité[i] + Bruit[i]$



CARACTÉRISATION DES SÉRIES TEMPORELLES

- **Comportement stationnaire**
- Parfois, les séries temporelles présentent un comportement stationnaire, i.e., son comportement est essentiellement le même, peu importe à partir de quel point on observe
 - Un random walk est stationnaire
 - Une série avec tendances et/ou saisonnalité n'est pas stationnaire
- Les processus stationnaires sont parfois les plus difficiles à modéliser car ils n'ont pas des motifs évidents



APPROCHES CLASSIQUES (NON-IA)

- Algorithmes d'Auto-régression

- Comme dans une régression linéaire, les modèles d'auto-régression font des prédictions à partir d'un nombre p de données historiques

- $T_t = a_0 + a_1T_{t-1} + A_2T_{t-2} + a_nT_{t-n}$

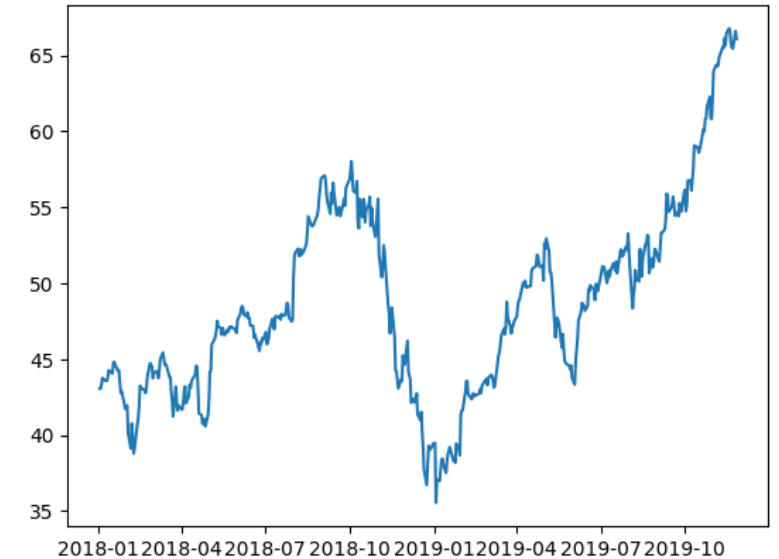
- Les coefficients obtenus sont relatifs à la dernière observation

- On peut prolonger la prédiction mais elle sera décorrélée des mesures
- Autre option, re-entraîner le modèle à chaque nouvelle observation



APPROCHES CLASSIQUES (NON-IA)

- **Algorithmes autorégressifs intégrant la moyenne mobile (ARIMA)**
 - ARIMA est une forme avancée des modèles AR
 - Inclut une dépendance entre l'observation et l'erreur résiduelle de la moyenne mobile des observations précédentes
- **Algorithmes autorégressifs saisonniers intégrant la moyenne mobile (SARIMA)**
 - SARIMA rajoute l'impact de la saisonnalité à ARIMA
 - C'est l'un des modèles les plus avancés
- **Algorithmes Lissage Exponentielle Holt-Winter**
 - Ces modèles utilisent le lissage pour garder une mémoire des variations de tendance et de saisonnalité



Modèle	Valeur prédite (27/11/2019)	Valeur réelle
AR	66.62344	66.07250
ARIMA	66.73182	66.07250
SARIMA	66.60217	66.07250
Holt-Winter	66.59249	66.07250

APPROCHES CLASSIQUES (NON-IA)

- **Avantages des solutions classiques**

- Très utilisées
- Peuvent obtenir des bons scores sur plusieurs types de données

- **Inconvénients**

- Cherchent des relations linéaires
 - incapables de trouver des motifs complexes non-linéaires
- Observation sur un historique fixe (lag)
- Ne fonctionnent qu'avec des séries univariées
 - les problèmes réels ont souvent plusieurs variables d'entrée
- Les prédictions se font sur un seul pas ($t+1$)
 - Pour des prédictions à long terme, on doit réitérer l'entraînement

EST-CE QU'UN MLP PEUT ÊTRE UTILISÉ ?

- Avant même de penser à des modèles plus élaborés, on peut essayer un simple modèle MLP
 - Préparer le dataset
 - Créer un modèle simple (par exemple, 3 couches totalement connectées)
 - Comparer avec un modèle classique
 - Lissage exponentiel Holt-Winter (HWSE)
- Attention : comparer (toujours) avec des solutions bêtes et simples
 - Un modèle de persistance (dummy) qui reproduit la dernière sortie
 - Un modèle d'interpolation entre les X dernières valeurs

PRÉPARATION DU DATASET

- Créer le dataset sous la forme d'une **fenêtre glissante**

```
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def get_time_series_datasets(features, ts_len):
```

```
    ts = get_time_series_data(ts_len)
```

```
    X = []
```

```
    Y = []
```

```
    for i in range(features + 1, ts_len):
```

```
        X.append(ts[i - (features + 1):i - 1])
```

```
        Y.append([ts[i]])
```

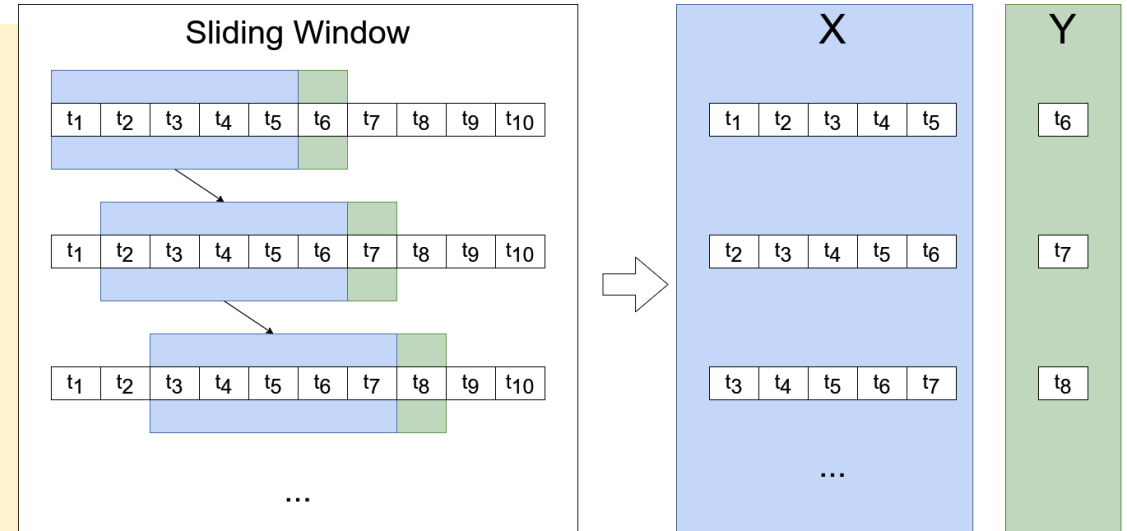
```
    X = np.array(X, dtype=np.float32)
```

```
    Y = np.array(Y, dtype=np.float32)
```

```
    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3, shuffle=False)
```

```
    X_val, X_test, Y_val, Y_test = train_test_split(X_test, Y_test, test_size=0.5, shuffle=False)
```

```
    return X_train, X_val, X_test, Y_train, Y_val, Y_test
```



DÉFINITIONS DES MODÈLES

- Le modèle classique (HWES)

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import numpy as np
from tensorflow import keras

class HwesPredictor(keras.Model):
    def call(self, x):
        # x : tf.Tensor or np.ndarray (shape: batch, time_steps)
        x_np = x.numpy() if hasattr(x, "numpy") else np.asarray(x)
        last_values = []
        for r in x_np:
            model = ExponentialSmoothing(r)
            results = model.fit()
            forecast = results.forecast()
            last_values.append([forecast[0]])
        return np.array(last_values, dtype=np.float32)
```

DÉFINITIONS DES MODÈLES

- Un modèle DNN (fully connected) à trois couches

```
from tensorflow import keras
from tensorflow.keras import layers

def build_fcnn_sequential(dim_input, neurones_L1, neurones_L2, dim_out):
    model = keras.Sequential([
        layers.Dense(neurones_L1, activation='relu', input_shape=(dim_input,)),
        layers.Dense(neurones_L2, activation='relu'),
        layers.Dense(dim_out) # Pas d'activation finale
    ])
    return model
```

ENTRAÎNEMENT DU MODÈLE CNN

```
random.seed(1)
tf.random.set_seed(1)

features = 256
ts_len = 3_000

x_train, x_val, x_test, y_train, y_val, y_test = get_time_series_datasets(features, ts_len)

net = build_fcnn_sequential(dim_input=features, neurones_L1=64, neurones_L2=32, dim_out=1)
net.compile(optimizer=keras.optimizers.Adam(),
            loss=keras.losses.MeanSquaredError(),
            metrics=[keras.metrics.MeanSquaredError()])

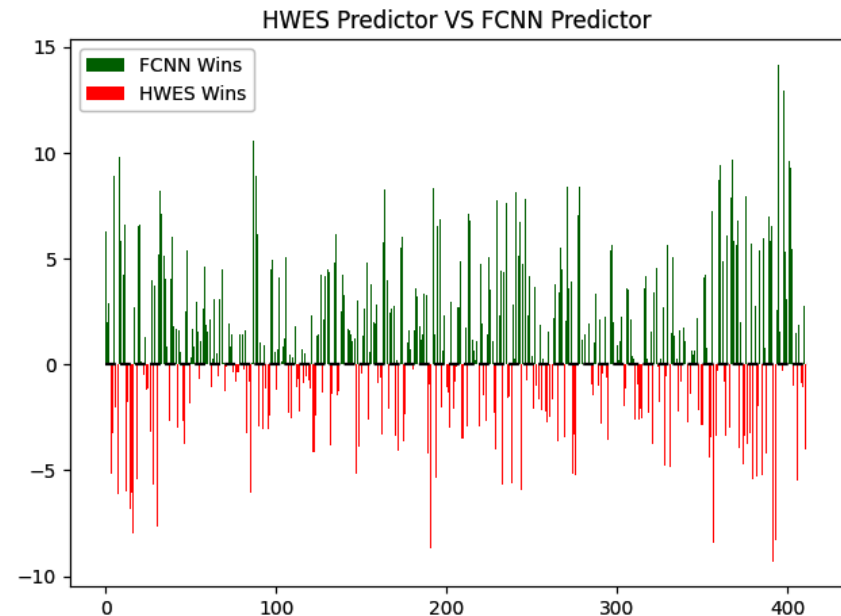
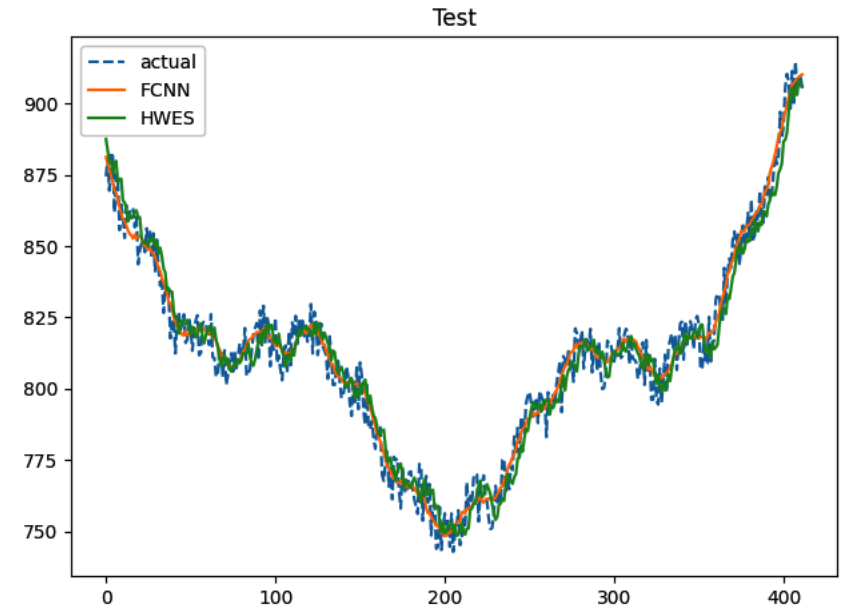
# Entraînement du modèle
history = net.fit(x_train, y_train,
                 epochs=10000, batch_size=len(x_train), validation_data=(x_val, y_val), verbose=1)

# Utilisation du réseau entraîné en mode "évaluation" (implicite après fit)
y_pred = net.predict(x_test)
```

COMPARAISON SIMPLE

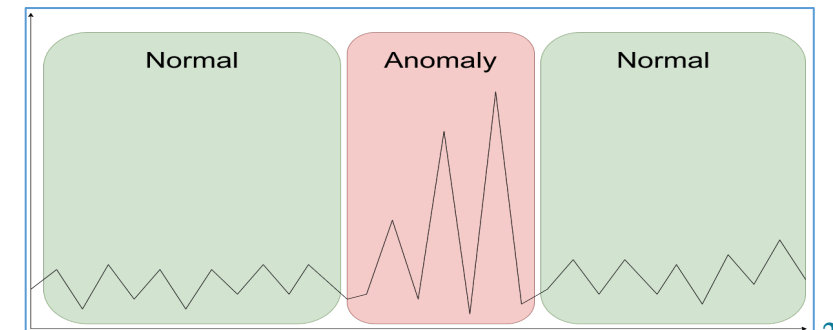
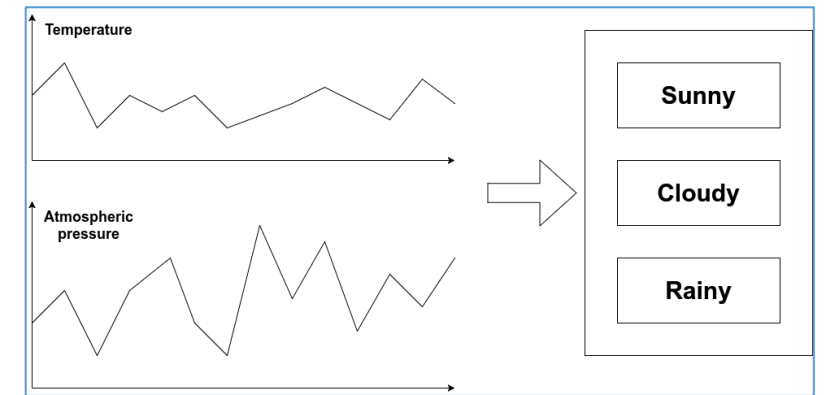
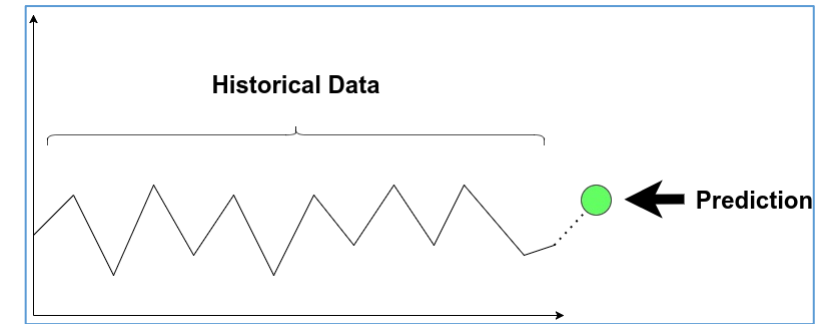
- Résultats :
 - Déjà, HWES et DNN font mieux que Dummy
 - La différence entre HWES et DNN n'est pas évidente sur le premier graphique
 - Le deuxième graphique montre qui a été plus performant à chaque coup, et de combien il dépasse l'autre modèle
 - Le vert (DNN) gagne plus souvent
 - HWES, quand il gagne, a un écart plus petit

Modèle	Perte (loss)
Dummy	75.77690
Interpolation	220.0586
HWES	59.23703
DNN	38.09475



TS EN TANT QUE PROBLÈME DEEP LEARNING

- Comme nous avons vu précédemment, les séries temporelles se prêtent à plusieurs usages
 - Modélisation
 - Forecast
 - Détection d'anomalies
- Les réseaux de neurones font essentiellement deux types d'opération
 - Régression
 - Classification
- Les applications doivent donc explorer ces deux modes d'opération

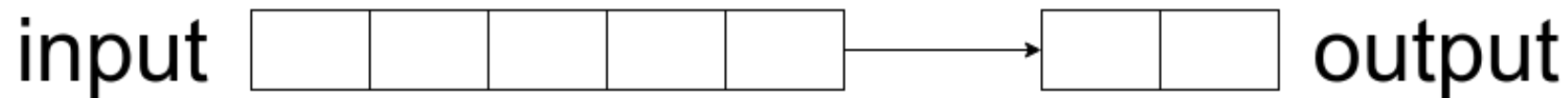


TS EN TANT QUE PROBLÈME DEEP LEARNING

- Contrairement aux approches classiques, le Deep Learning peut traiter des modèles univariés et multivariés

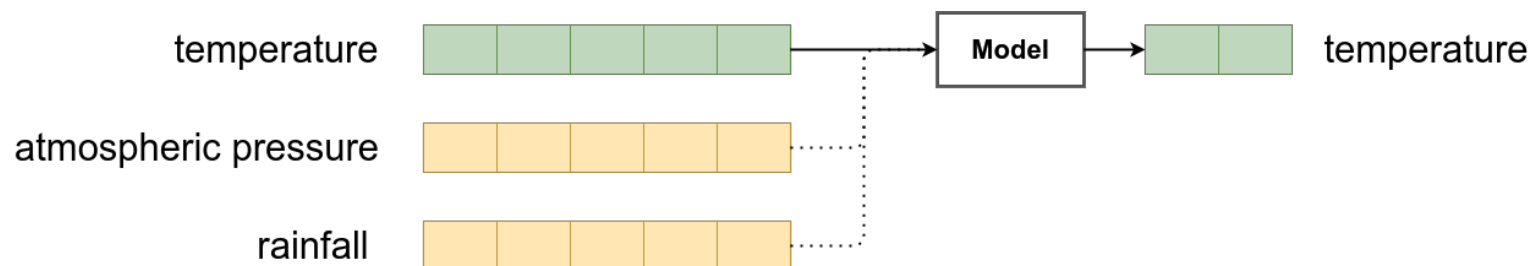
- **Univariate input – univariate output**

- Le modèle assume un seul type de série temporelle en entrée et un autre en sortie
 - Seul modèle traité par ARIMA, SARIMA, HWES, etc.



- **Multivariate input – univariate output**

- C'est le modèle que révèle la puissance du deep learning : trouver des motifs complexes multivariés
 - Souvent, il y a une série temporelle d'entrée et sortie, plus des séries temporelles auxiliaires



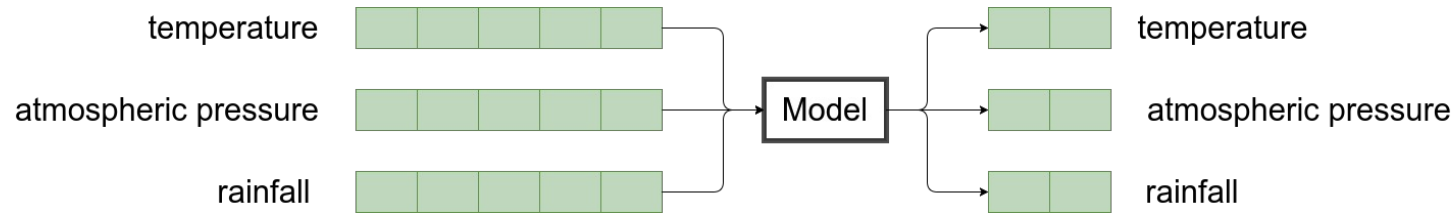
TS EN TANT QUE PROBLÈME DEEP LEARNING

- **Multivariate input - multivariate output**

- C'est le type le plus compliqué à développer. Il y a deux approches

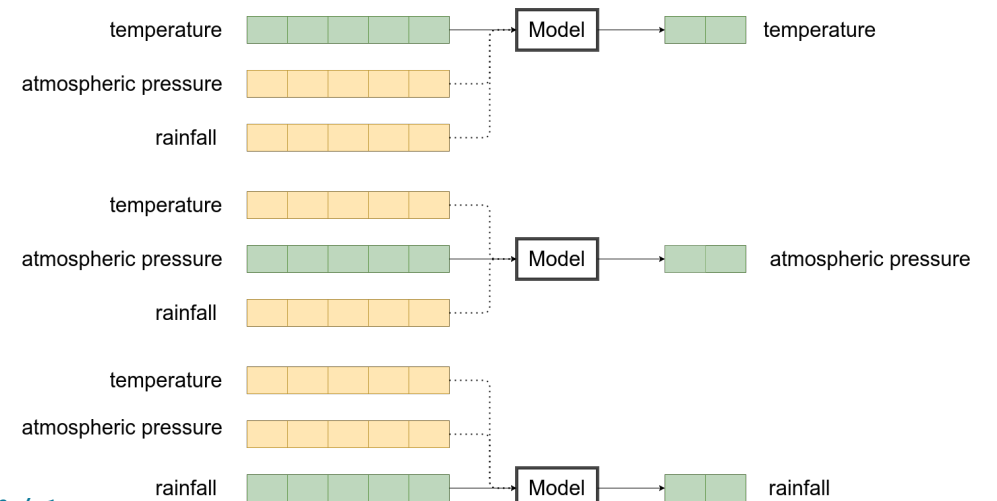
- Many to Many

- On développe un seul modèle capable de gérer toutes les variables prédictives et les sorties



- Many to One

- Autre option est de développer des modèles différents pour chacune des sorties requises

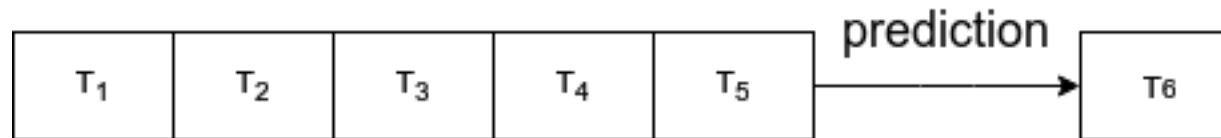


TS EN TANT QUE PROBLÈME DEEP LEARNING

- Les modèles traditionnels étaient aussi caractérisés par la capacité d'effectuer une seule sortie

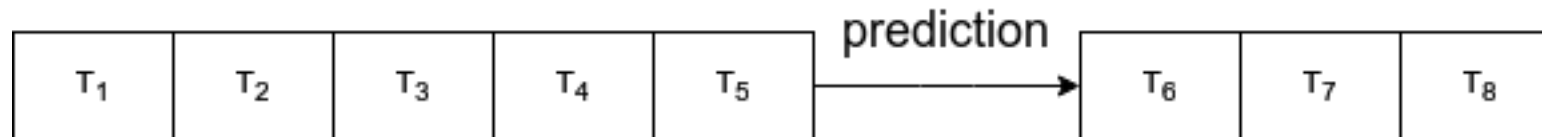
- Single-Step**

- Le modèle n'est capable de prédire qu'une seule valeur future



- Multi-Step**

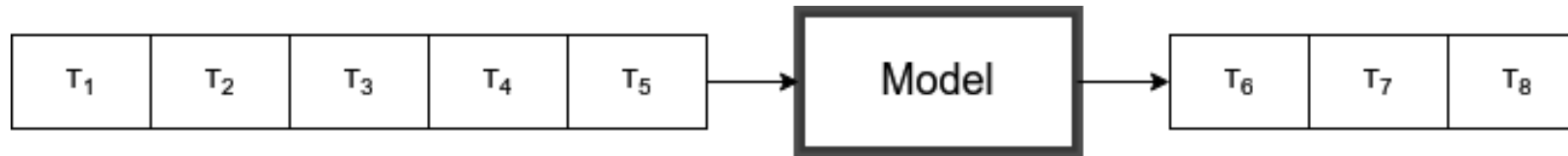
- Les modèles DL permettent également la prédiction de valeurs pour plusieurs étapes futures
 - Evidemment, la précision décline à chaque étape future
 - Trois approches** peuvent être utilisées pour créer des modèles multi-step



TS EN TANT QUE PROBLÈME DEEP LEARNING

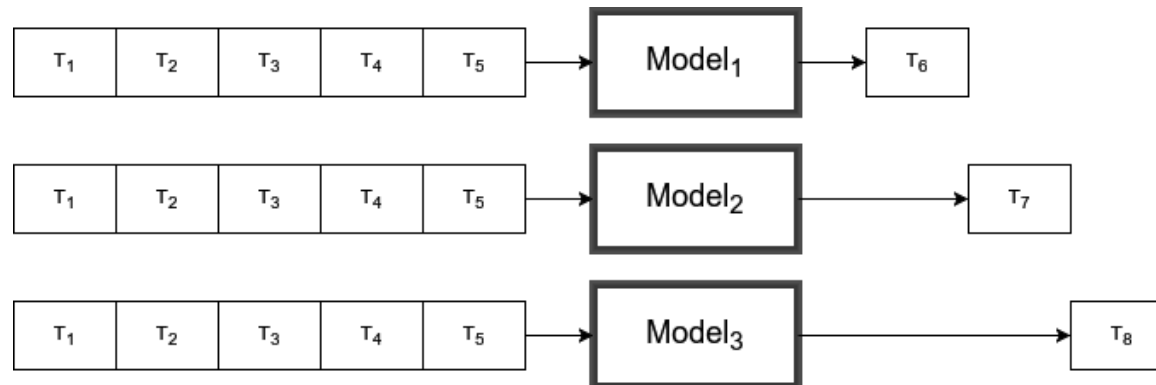
- **Single multi-step**

- Ce modèle assume une sortie multiple
- Facile de maintenir et déployer mais difficile d'entraîner



- **Multiple single-step**

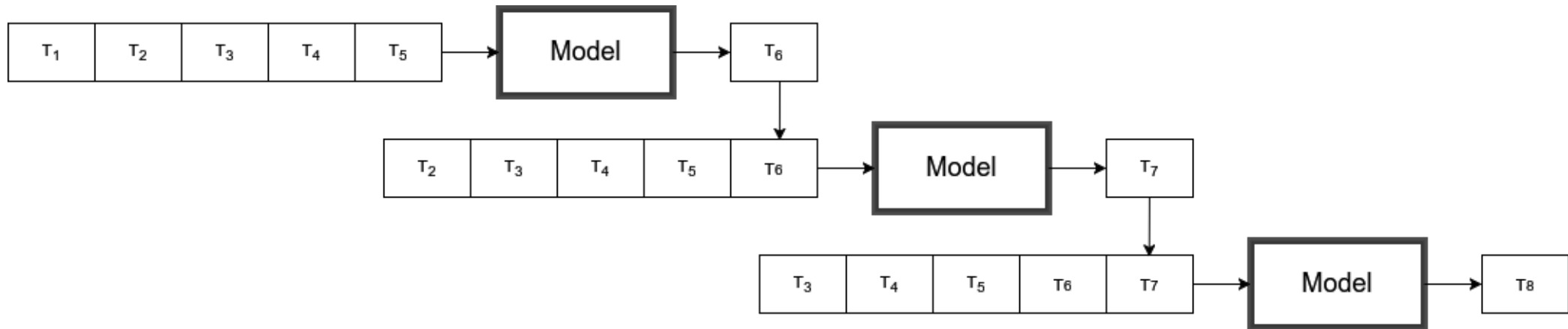
- Chaque étape de sortie a son modèle
- Plus facile à entraîner et permet en plus d'avoir des architectures différentes pour chaque modèle



TS EN TANT QUE PROBLÈME DEEP LEARNING

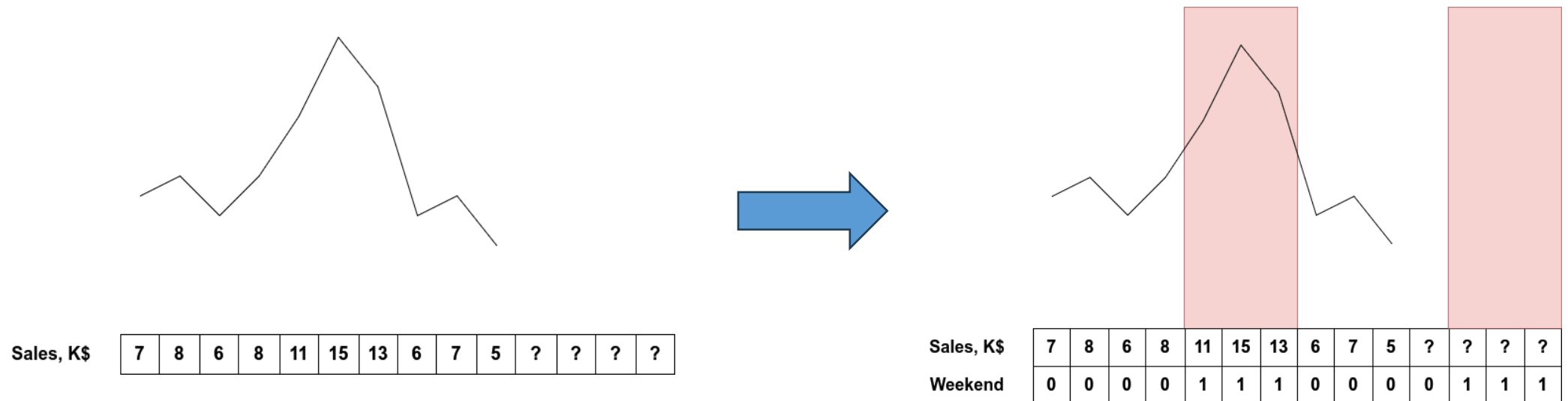
- **Single-step récursif**

- Le modèle plus simple d'entraîner et d'implémenter
- Chaque sortie est le résultat d'une exécution "décalée" avec une partie des sorties précédentes
- Avantage majeur : le nombre d'étapes futures n'est pas fixé, tout dépend du nombre de récursions



ADAPTER LES DATASETS – FEATURE ENGINEERING

- Si le DL nous donne la possibilité de faire du "multivariate input", il peut être intéressant d'enrichir les données
 - Sur une série de ventes d'un magasin, une variation saisonnière est attendue
 - Plus de ventes le weekend
 - Mais le modèle n'a pas cette information, il ne sait pas le "contexte" de fonctionnement d'un magasin
 - On peut l'aider en donnant des entrées supplémentaires pour l'aider à trouver cette corrélation



ADAPTER LES DATASETS – FEATURE ENGINEERING

- Parfois on peut ajouter des séries temporelles issues de transformations mathématiques sur les données originales
 - Ces données apportent des nouvelles informations
 - Ex : l'élan du marché des actions, avec le nombre de jours consécutifs d'expansion et rétraction

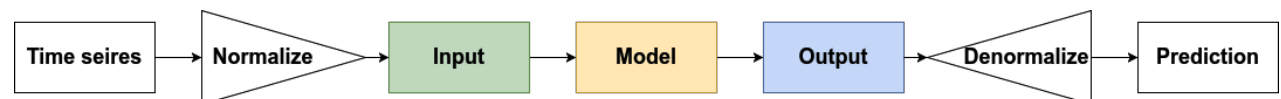
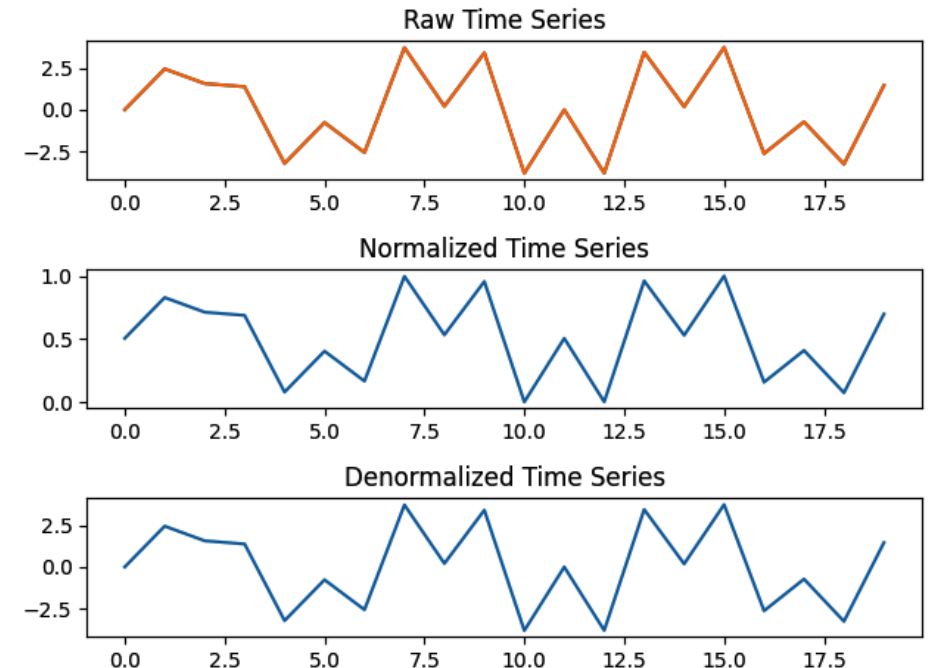
Close Price	121	125	127	130	135	133	129	120	117	120
Feature Engineering										
Close Price	121	125	127	130	135	133	129	120	117	120
Sequence Length	0	1	2	3	4	-1	-2	-3	-4	1

ADAPTER LES DATASETS – PRÉ ET POST-PROCESSING

- En dehors de la *feature engineering*, il y parfois des transformations utiles à faire sur les données

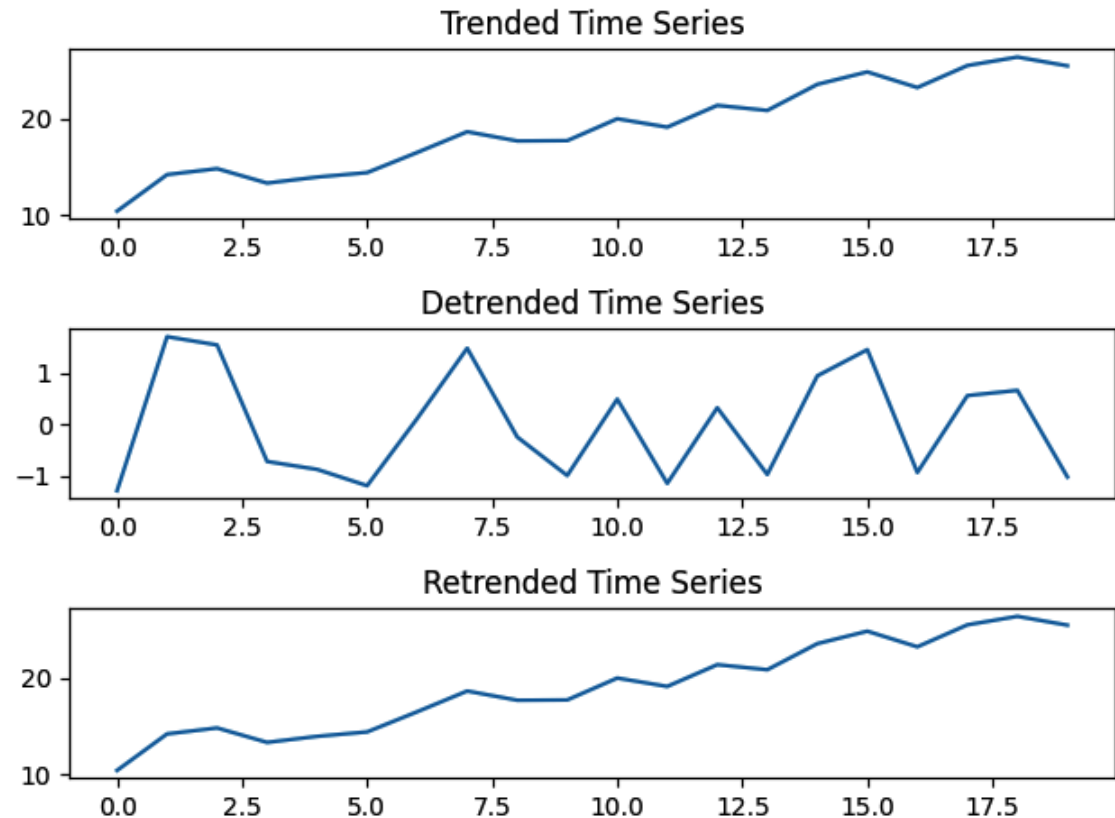
- **Normalisation**

- Opération qui vise à **ramener les données sur des intervalles $[0, 1]$ ou $[-1, 1]$**
- Souvent, apporte plusieurs avantages
 - Réduit les différences d'échelle entre deux variables
 - (ex : prix des maisons vs nombre de transactions)
 - Recentre les données (skew)
 - Permet de mieux explorer la précision des Float32 et Float16 (format *signe-exposant-mantisse*)
- Plusieurs bibliothèques (sklearn, etc.) font de la normalisation
- Attention, la normalisation doit être suivi d'une **dénormalisation** à la sortie



ADAPTER LES DATASETS – PRÉ ET POST-PROCESSING

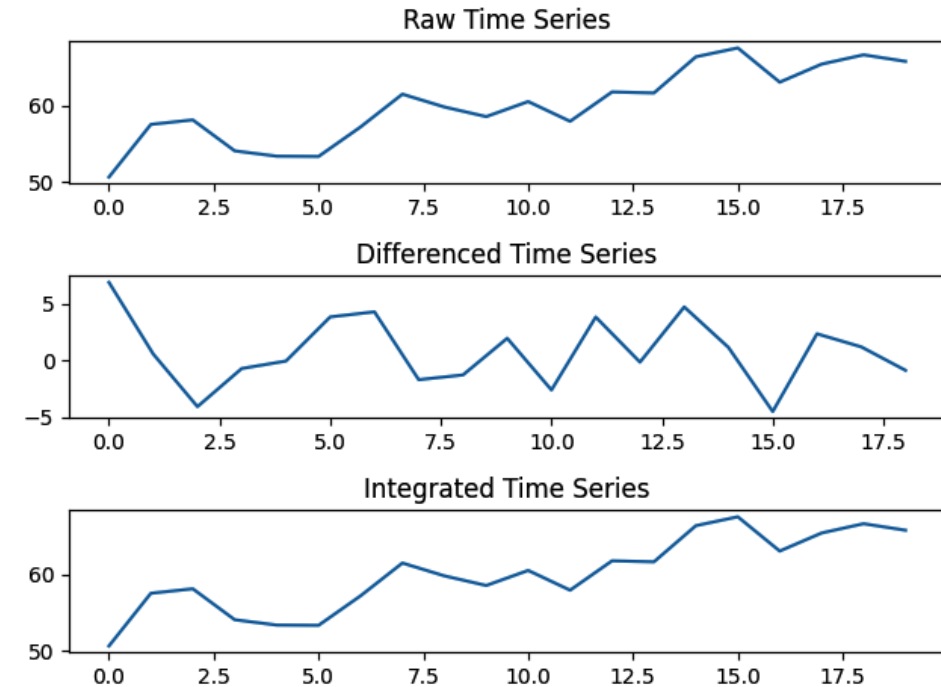
- **Suppression des tendances**
 - Simplifie le modèle
 - Plus besoin de modéliser la fonction F
 - Aide à normaliser les données
 - Salaires de 1950 << salaires de 2024
 - Fait ressortir d'autres variations qui peuvent augmenter la précision
- L'implémentation est moins automatique
 - Détermination de la tendance
 - Régression linéaire, par exemple
 - Transformation des données
 - Valeur – régression linéaire



ADAPTER LES DATASETS – PRÉ ET POST-PROCESSING

- **Différentiation**

- Autre manière d'éliminer la tendance
- Permet de supprimer des tendances permanentes ou **temporaires**
- Se concentre sur les variations à chaque pas
 - $D_t = T_t - T_{t-1}$

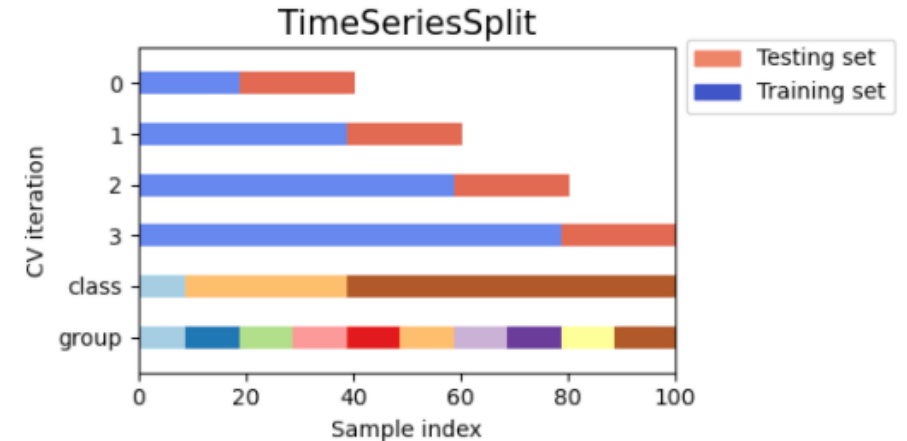


ADAPTER LES DATASETS – DÉCOUPAGE

- Dernier point :

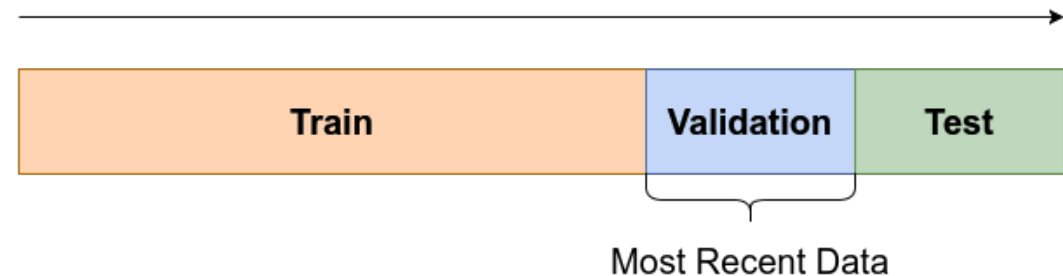
- **ne jamais faire du shuffle ni de la cross-validation**

- La cross-validation casse le lien temporel
 - On peut apporter de la variabilité en augmentant progressivement la taille des groupes train/validation
 - En sklearn il y a la fonction `TimeSeriesSplit()` qui génère des ensembles train/test



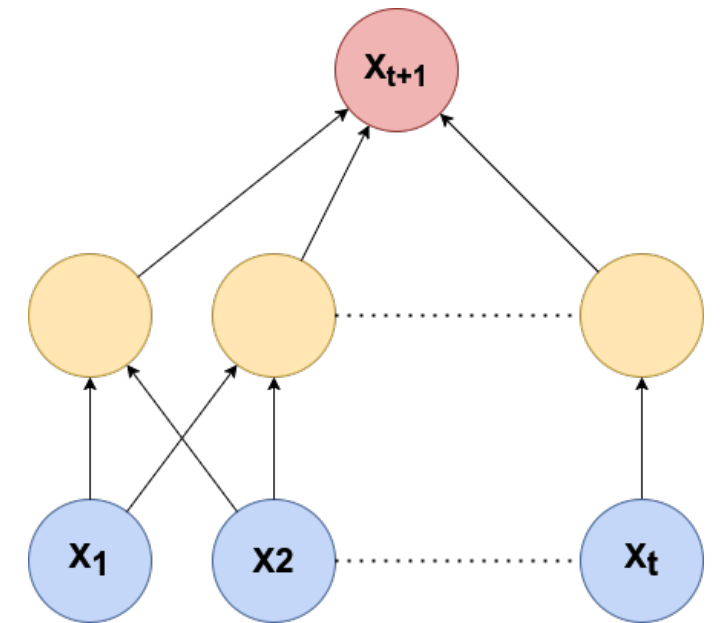
- **Entraîner avec les données plus anciennes, valider avec les**

- Et garder des données encore plus récentes pour confirmer le choix des modèles (test)



SÉRIES TEMPORELLES AVEC DES CNN/DNN

- On a vu qu'il est possible de faire des prédictions avec les réseaux denses
- Il est tout à fait possible de faire avec des CNN
 - On verra en TP (températures à Londres)
- **Attention : les DNN/CNN ont des inconvénients**
 - Entrée de taille fixe
 - Si les vecteurs d'entrée ont des données manquantes, il est impossible d'utiliser les modèles
 - **Ne perçoivent pas les vecteurs comme des séquences ordonnées**
 - Ce sont juste des chiffres
 - Pas d'effort pour trouver un motif lié à l'ordre des données
- Les **Réseaux Récurrents** apportent une solution à ces problèmes
 - Le concept d'état caché (hidden state)

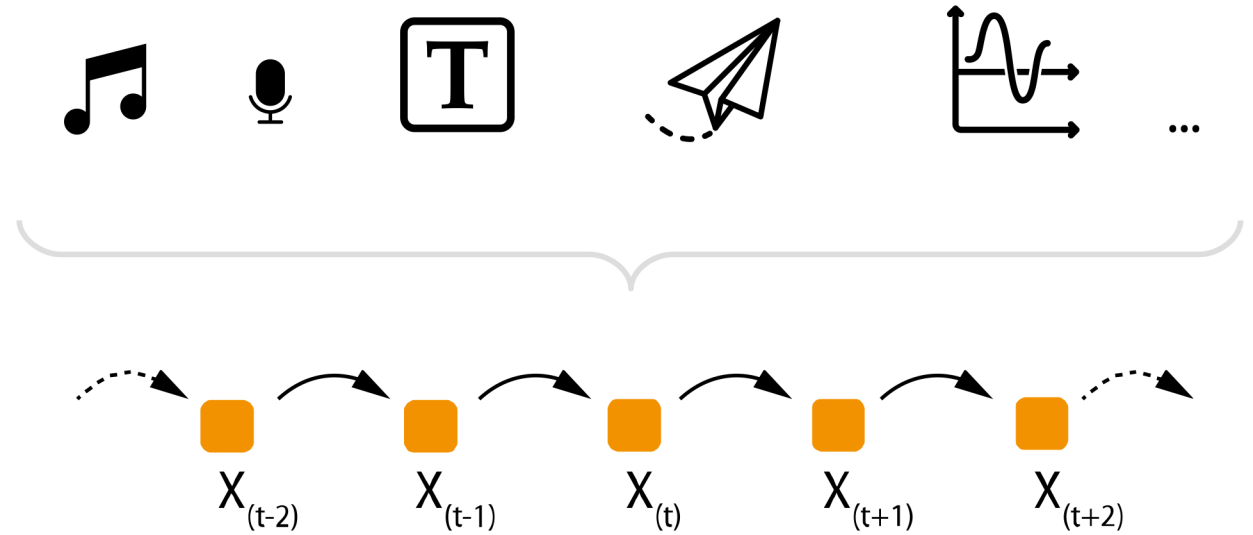




Recurrent Neural Networks - RNN

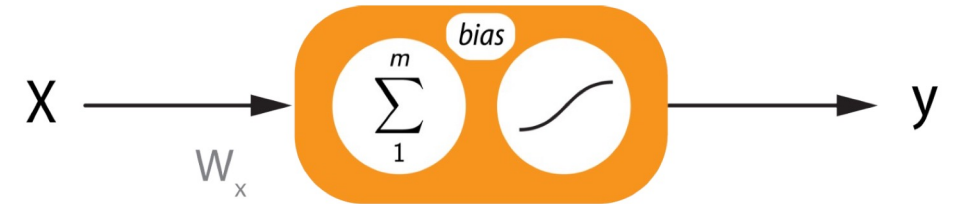
RÉSEAUX DE NEURONS RÉCURRENTS

- RNNs sont utiles lorsque l'ordre/arrangement compte
 - Un besoin important pour les séries de données temporelles
- Plusieurs usages
 - Prédiction de séries temporelles
 - Génération de textes
 - Compléter une phrase
 - Traductions
 - Description d'une scène



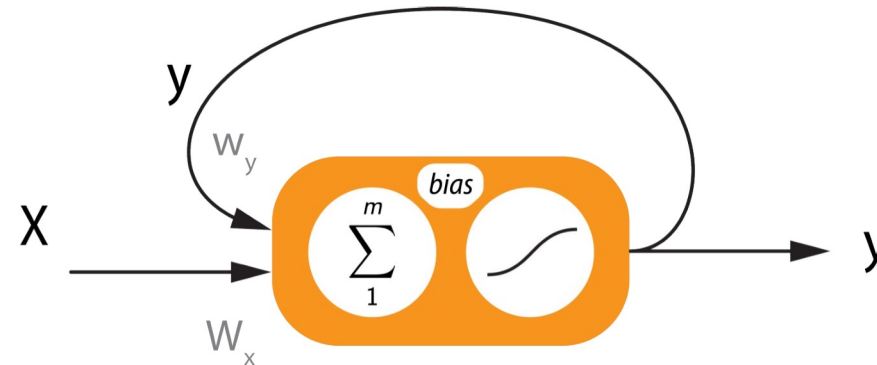
NEURONES CLASSIQUES VS NEURONES RÉCURRENTS

- Dans un neurone classique, la sortie dépend de l'entrée, des poids et du biais



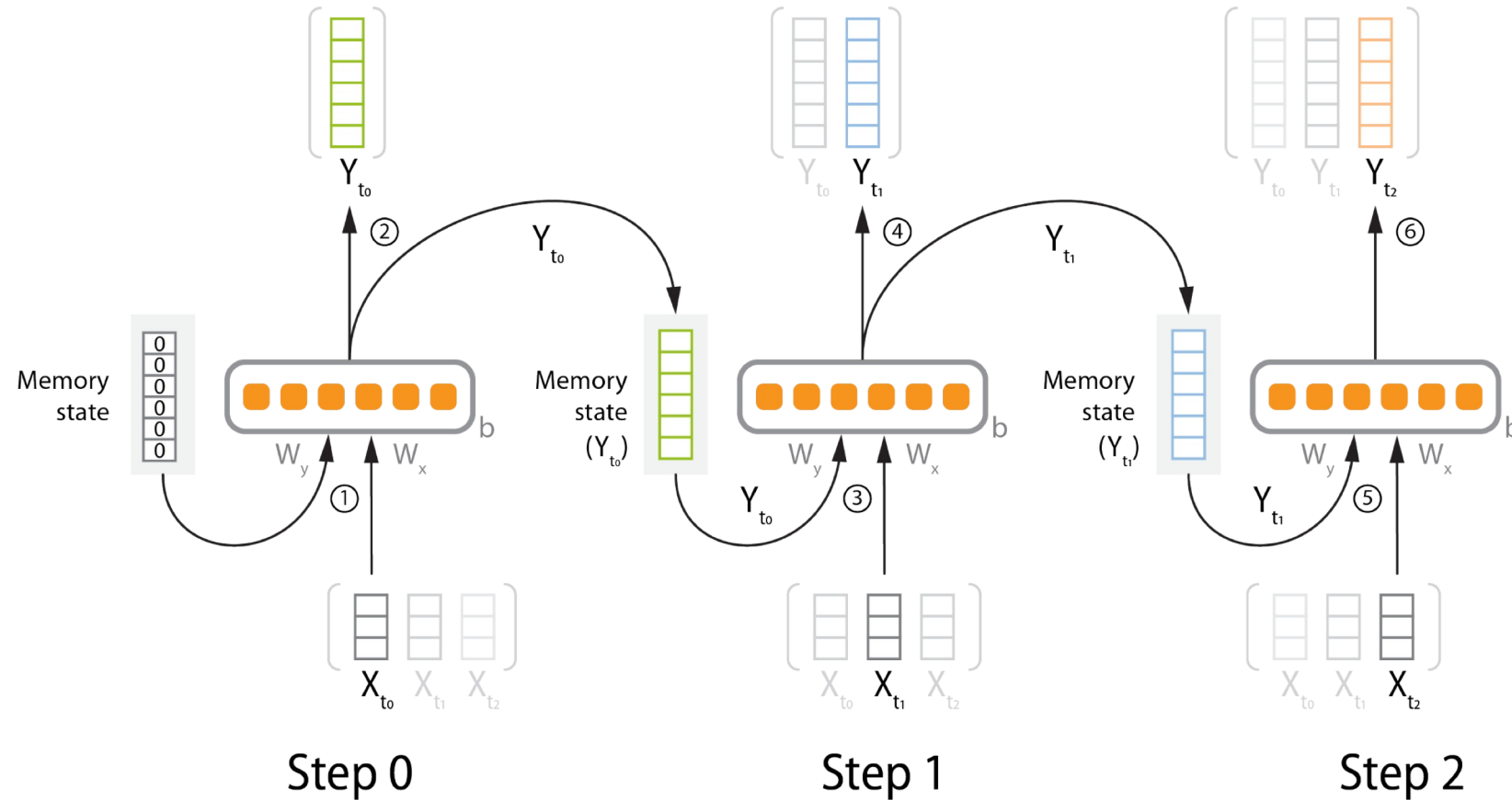
- Dans un RNN, la sortie est "reliée" à l'entrée

$$y = \sigma(W_x^T \cdot X + b)$$



$$y_{(t)} = \sigma(W_x^T \cdot X_{(t)} + w_y \cdot y_{(t-1)} + b)$$

UTILISER L'ENTRÉE COMME UN FLUX (STREAM)



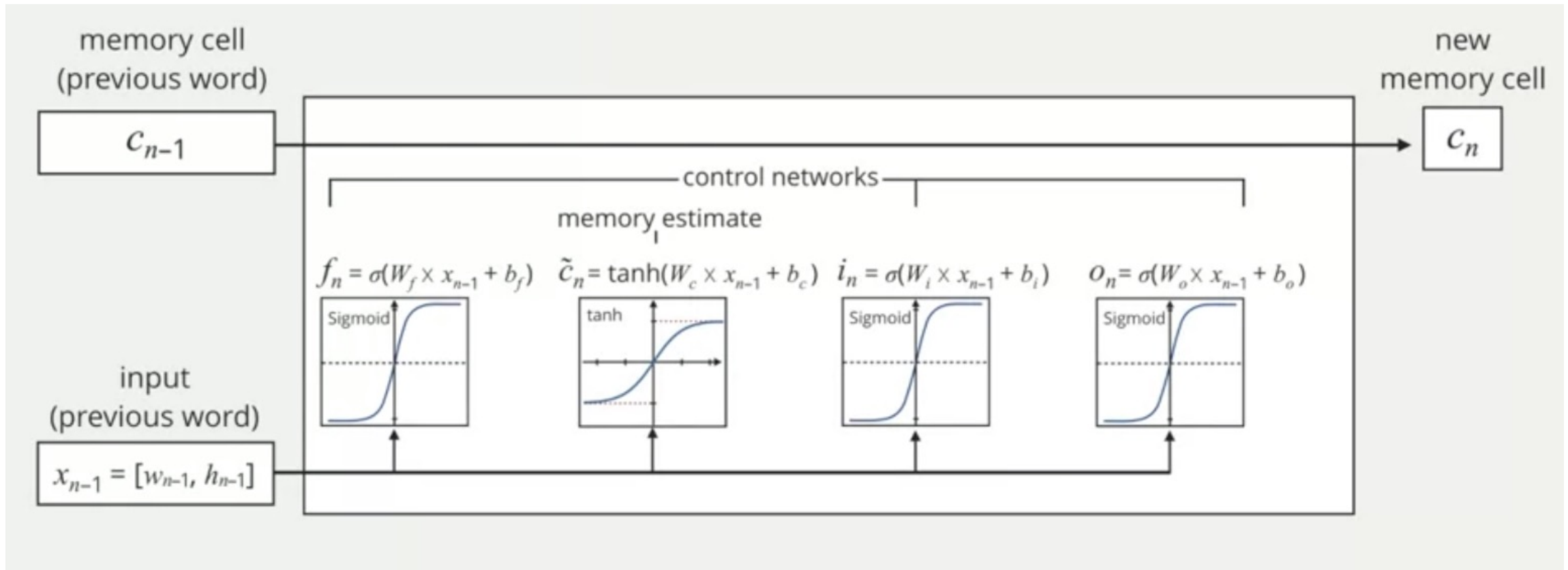
Inconvénient : la mémoire est écrasée à chaque tour, difficile de garder des concepts "distants"

POURQUOI UN RNN SIMPLE NE SUFFIT PAS ?

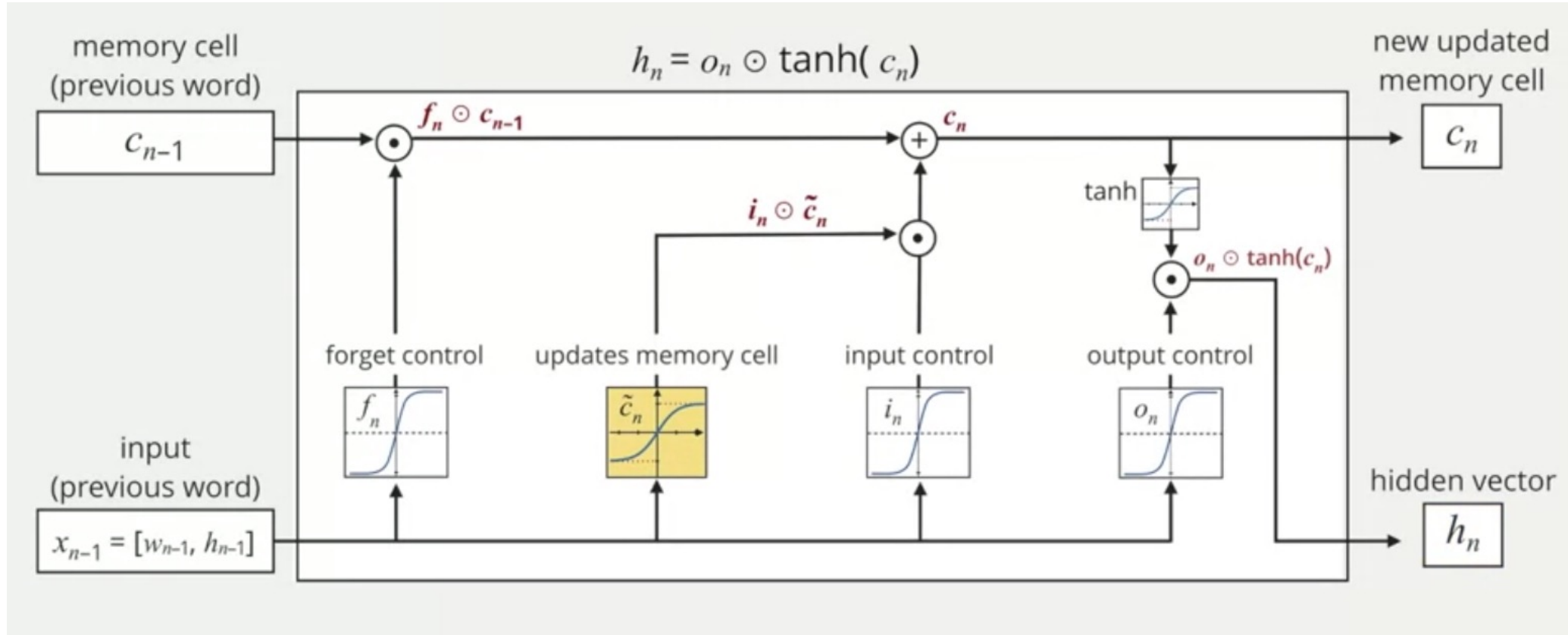
- RNNs doivent réalimenter l'entrée (feedback loop) pour garder l'information en mémoire
- Mais ils tendent à écraser cette information et finissent par ne pas la garder pendant trop longtemps
- Nous avons besoin d'un modèle qui contrôle quelle information doit être gardée et quelle information on peut oublier
- Le modèle **LSTM (Long Short-Term Memory) [1997]**
 - Plus récemment, le modèle GRU (Gated recurrent units) [2014]

INSIDE THE NEURAL NETWORK

- 4 réseaux de neurones différents!!!!
 - 3 réseaux de contrôle -> chacun entraîne avec ses propres poids - Tous utilisent la fonction *Sigmoid*
- Plus une "cellule mémoire" qui utilise la fonction *tanh*

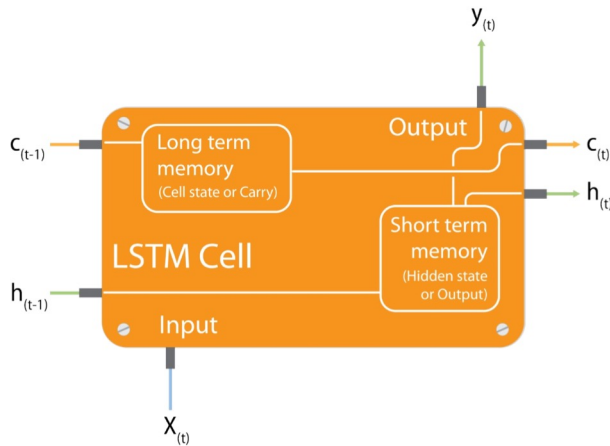


CHAQUE MODÈLE A UNE FONCTION



LSTM AVEC KERAS

- Keras inclut une boucle LSTM



Entrée =
tableau avec 32x 20 "pas temporels" avec 8 variables chacune)

```
inputs = tf.random.normal([32, 20, 8])
```

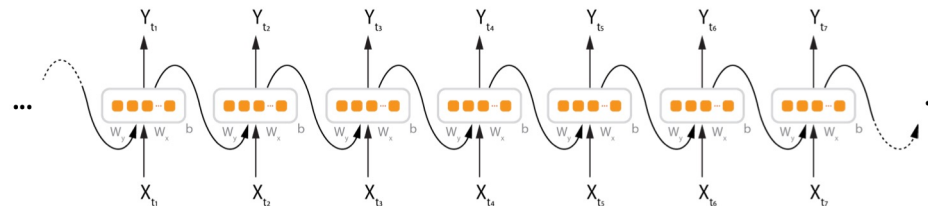
```
lstm = tf.keras.layers.LSTM(16)  
output = lstm(inputs)
```

16 LSTM =
16 récursions

Serie to **vector**

Inputs shape is : (32, 20, 8)
Output shape is : (32, 16)

Sortie = 32x la sortie de chacun des 16 LSTM en séquence



DES USAGES POUR LES LSTM/GRU

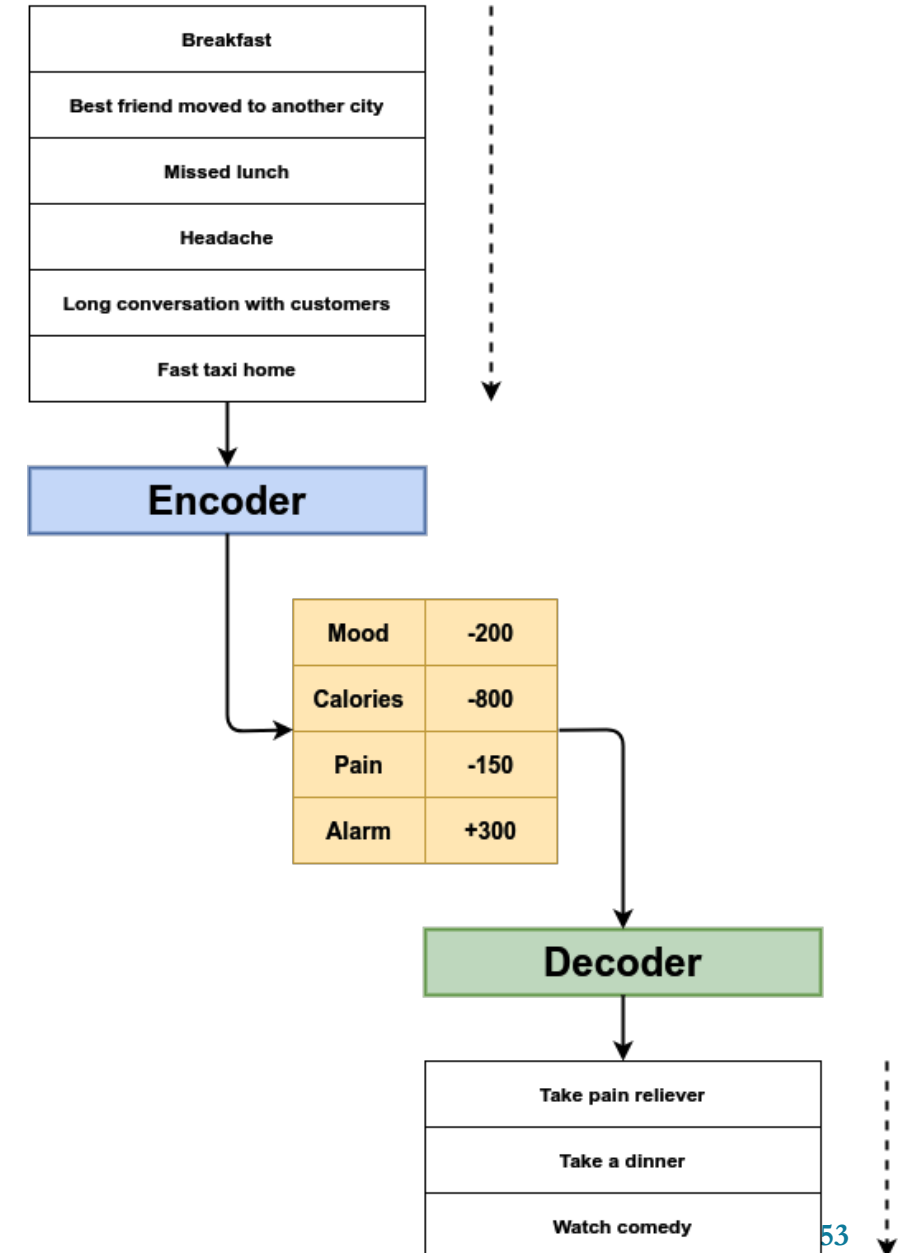
- Plusieurs types de problèmes peuvent être traités :
 - **One-to-One** : on trouve une entrée et une sortie uniquement
 - C'est le cas par exemple d'une image en entrée, où il faut sortir un label unique
 - **Many-to-One** : une séquence de données est fournie en entrée, afin de prédire une seule valeur
 - C'est le cas d'une série de températures, où il faut prédire la prochaine valeur
 - **One-to-Many** : une seule entrée donne origine à plusieurs sorties
 - C'est le cas d'une image en entrée, qui aura une description (séquence de mots) en sortie
 - **Many-to-Many** : des séquences sont fournies en entrée et on obtient également des séquences
 - Ex : prédire le comportement de la bourse de valeurs dans les prochains 3 jours, à partir des 7 derniers jours
 - Ex : fournir une phrase en entrée et avoir sa traduction en sortie

MODÈLES AVANCÉS

- Si les modèles GRU et LSTM sont très efficaces, d'autres modèles récents apportent des nouveautés
 - Les modèles **encodeur-décodeur**
 - Même principe des encodeurs-décodeurs pour les images
 - L'encodeur va créer le hidden state afin de retenir les informations importantes
 - Le décodeur va interpréter le hidden state et estimer l'étape suivante
 - Les modèles **TCN (Temporal Convolution Networks)**
 - Utilise le principe de convolutions pour couvrir des longueurs plus importantes
 - Garder les informations importantes
 - Réduire le coût de stockage/calcul
- Les **Transformers** sont également étudiés
 - leur coût est bien supérieur pour des résultats pas toujours intéressants

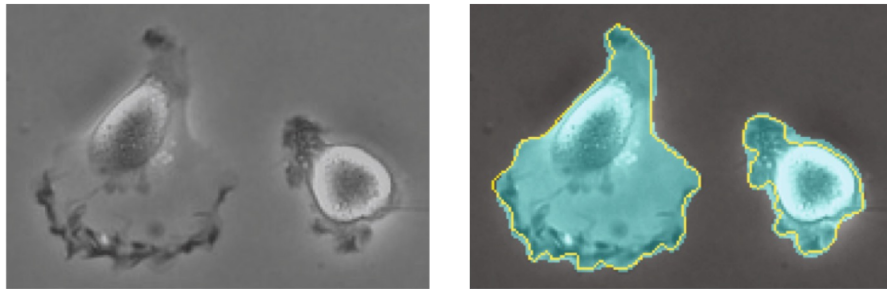
ENCODEUR-DÉCODEUR

- Même principe des encodeurs-décodeurs pour les images
 - L'encodeur va mettre à jour le hidden state afin de retenir les informations importantes
 - Le décodeur va interpréter le hidden state et estimer l'étape suivante
- Ce modèle reproduit le mécanisme de la mémoire et de la prise de décision d'une personne
 - On ne retient pas tout, mais certaines choses nous font prendre des décisions (parfois en cascade, selon un ordre de priorités)

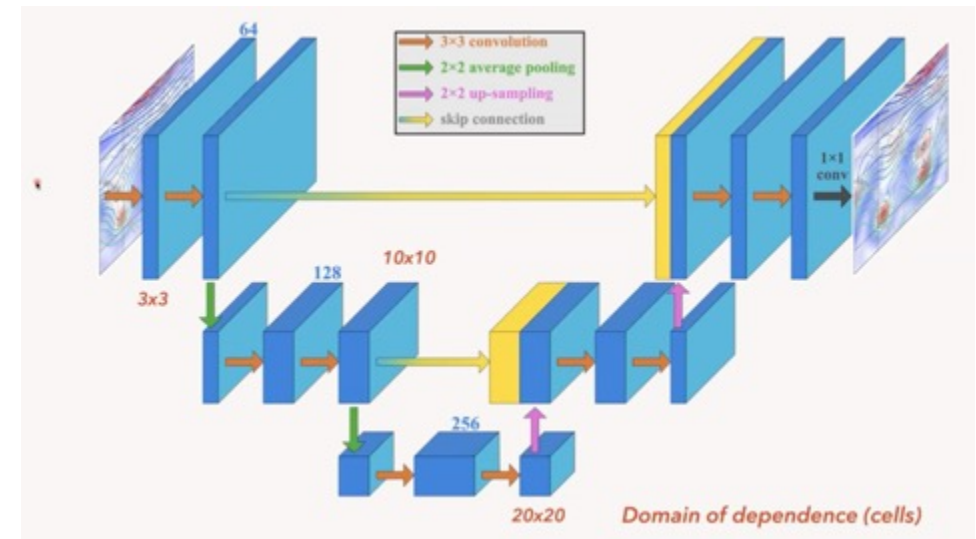


AUTRE EXEMPLE D'ENCODEUR-DÉCODEUR

- Lorsque les données forment des matrices, il est possible d'utiliser des techniques d'IA pour les images
 - Et pas seulement pour la classification/segmentation
- Modèle UNet (encodeur-décodeur avec connexions de saut)
 - Initialement créé pour segmenter les images médicales
 - L'entraînement consiste à montrer les images originales et les images « cibles »

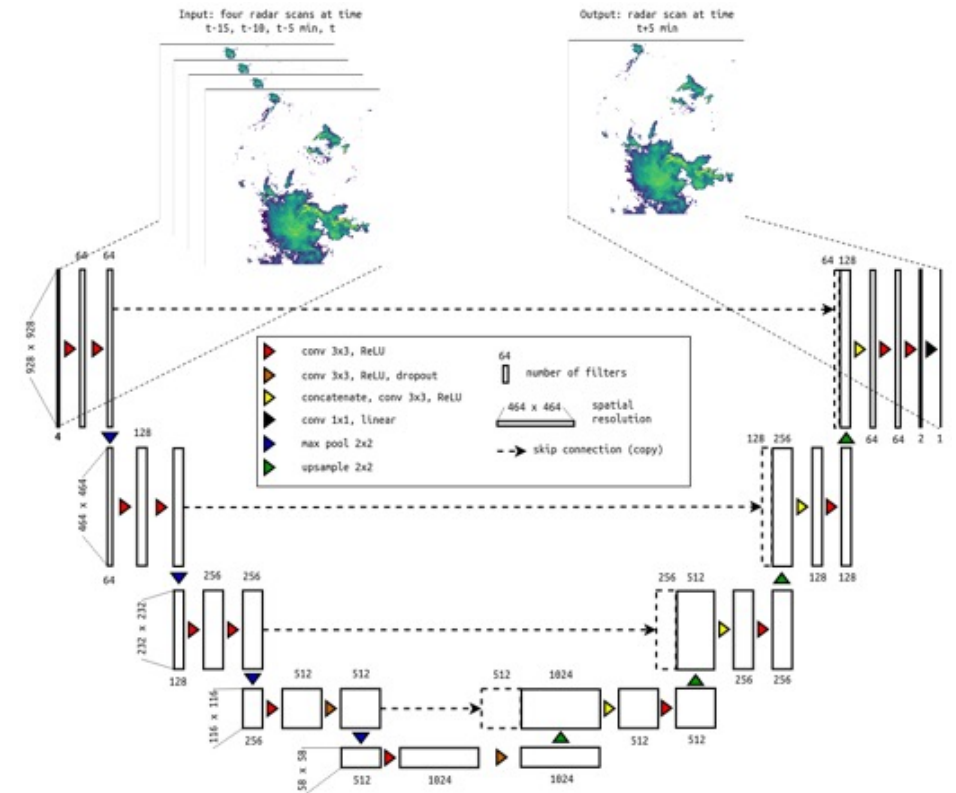


- Comment détourner son usage ?
 - Image RGB = 3 canaux pour les couleurs
 - Même "pas temporel"
 - Mais on peut aussi rajouter des canaux séquentiels



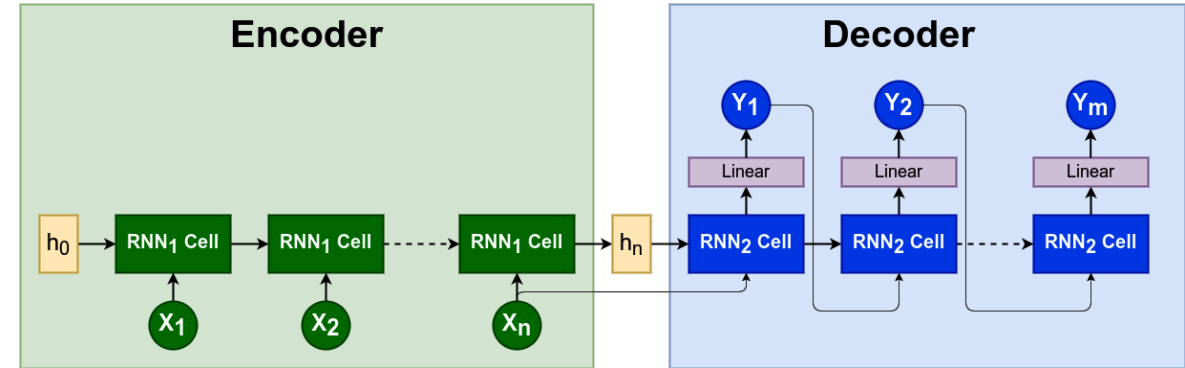
UNET POUR LE FORECAST RADAR (NOWCAST)

- Le modèle UNET peut également accepter plusieurs images d'entrée
- Il génère une seule sortie, mais il est possible de « faire glisser » la fenêtre pour obtenir davantage d'itérations
 - Aucune connaissance en géophysique n'est requise
- Également utilisé dans les modèles expérimentaux de l'ECWMF



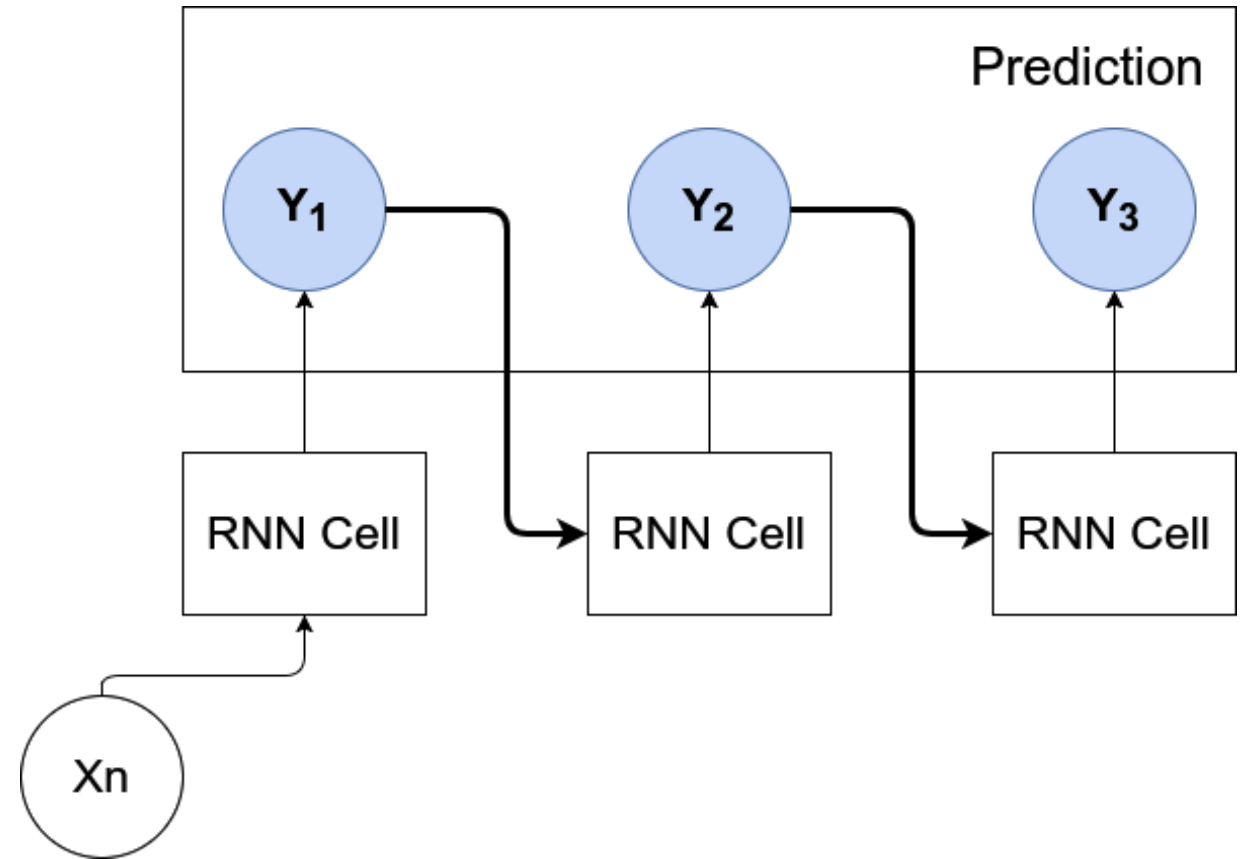
ENCODEUR-DÉCODEUR AVEC DES RNN

- Plutôt que compresser l'information, on peut l'encoder de façon récursive
- L'encodeur a une structure déjà connue
 - La même qu'on a vu dans GRU/LSTM
- LE décodeur peut aussi être récursif
 - Un décodeur peut donc fournir un nombre illimité de sorties
- De plus, l'encodeur et le décodeur peuvent être implémentés séparément
 - Ex : encodeur GRU, décodeur LSTM



ENTRAÎNER UN RÉSEAU ENCODEUR-DÉCODEUR

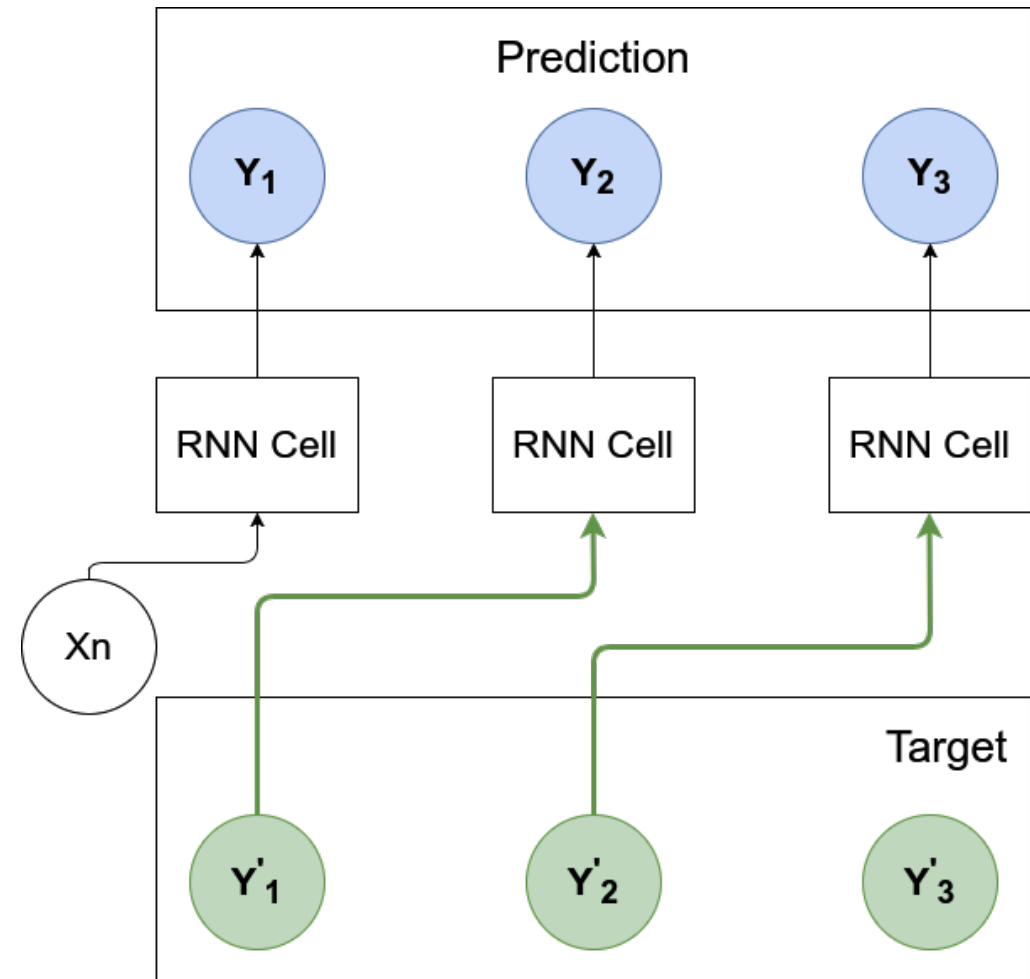
- Il y a trois approches pour entraîner un décodeur
 - Réursive**
 - Chaque entrée du RNN est le résultat de l'action précédente



ENTRAÎNER UN RÉSEAU ENCODEUR-DÉCODEUR

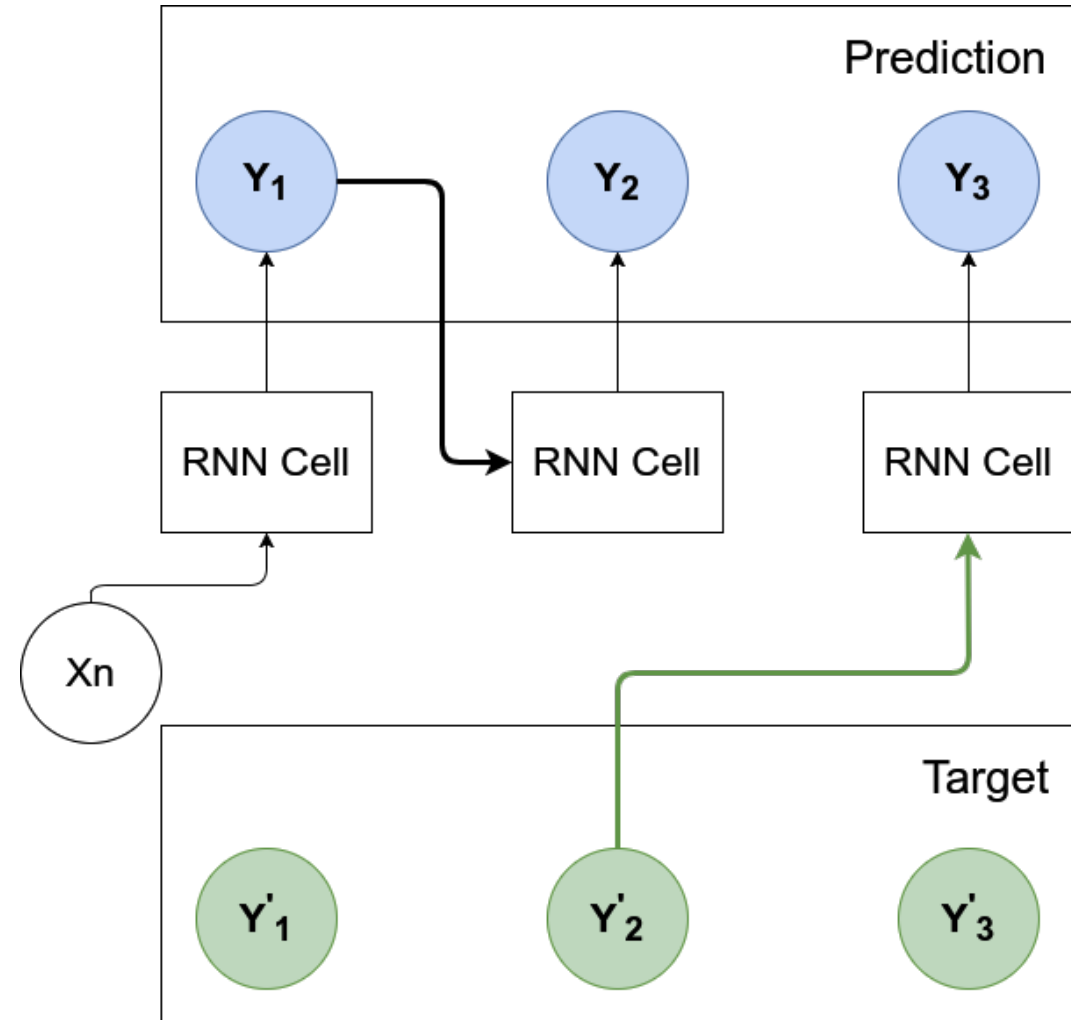
- **Teacher forcing**

- Les cellules RNN du décodeur reçoivent des valeurs cible lors de l'entraînement
- Ceci rajoute des informations extérieures à l'entraînement avant le calcul de la perte
- Renforce les prédictions à long terme



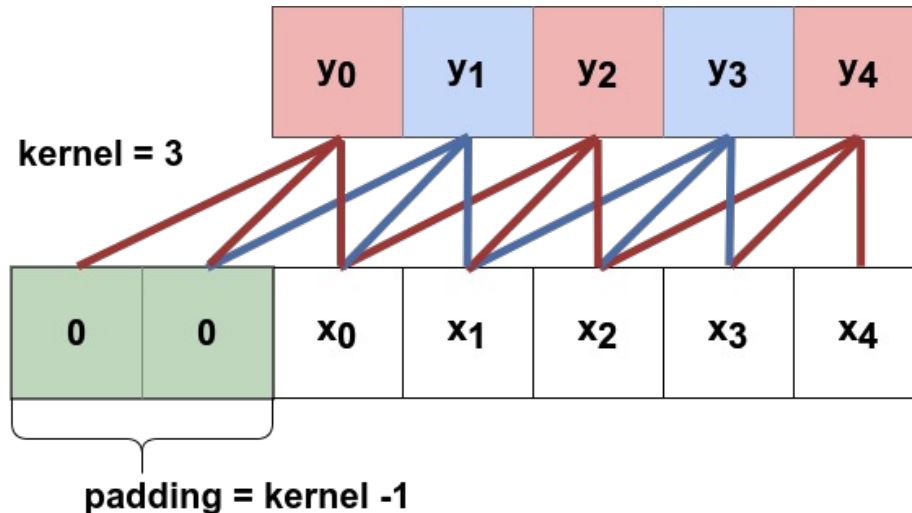
ENTRAÎNER UN RÉSEAU ENCODEUR-DÉCODEUR

- **Mixed teacher forcing**
 - Combine les deux méthodes précédentes
 - Un ratio est passé en paramètre
 - En général, ça donne les meilleurs résultats



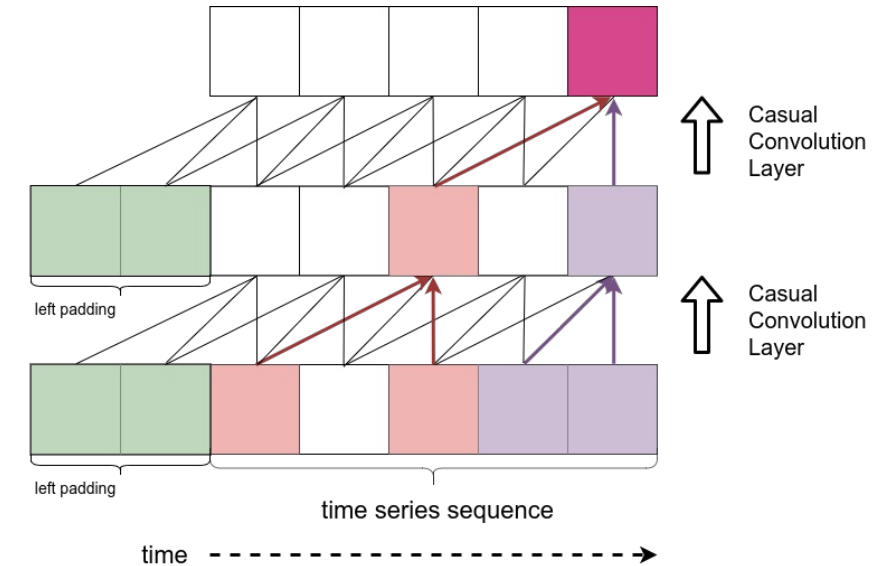
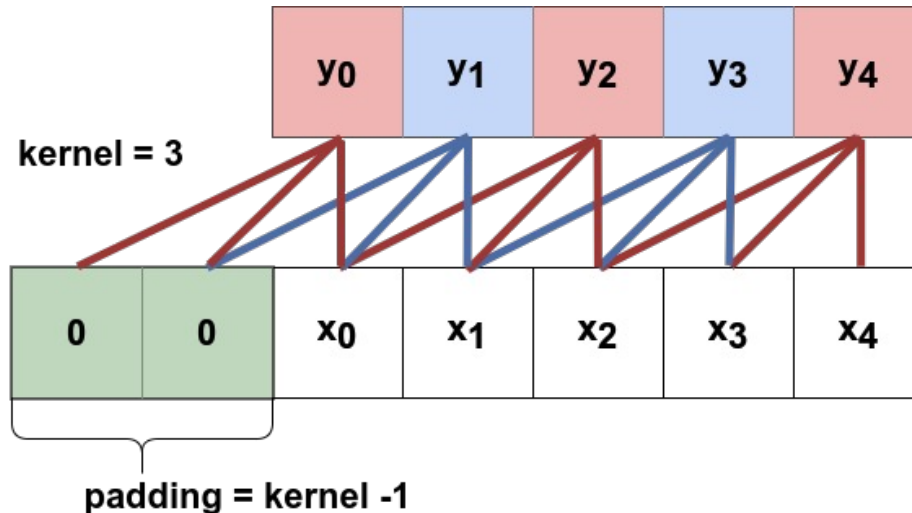
TEMPORAL CONVOLUTIONAL NETWORK (TCN)

- Principe :
 - Appliquer les concepts de convolution pour "résumer" les informations importantes
 - Appliquer le concept de **stride** à différentes échelles
- Convolution casuelle (Casual Convolution)
 - C'est essentiellement une convolution 1D
 - Utilisation de paddings pour garder la taille du vecteur



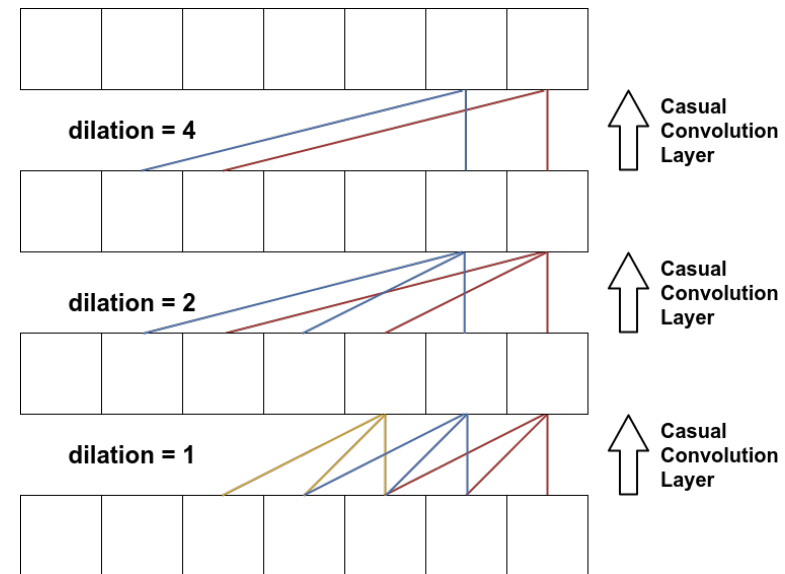
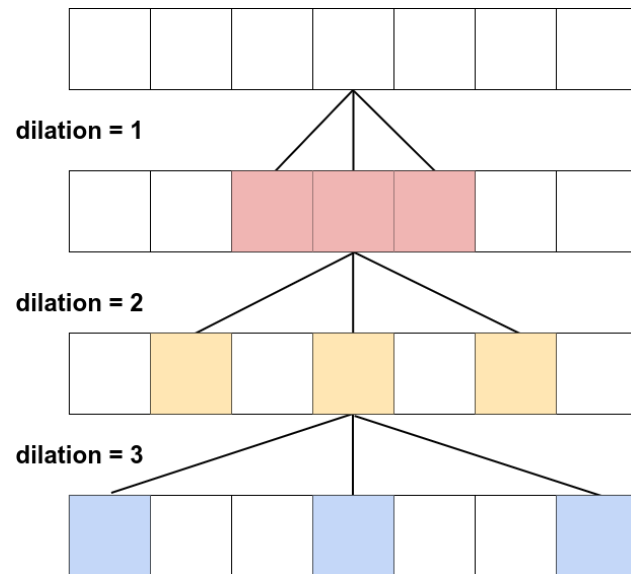
TEMPORAL CONVOLUTIONAL NETWORK (TCN)

- Principe :
 - Appliquer les concepts de convolution pour "résumer" les informations importantes
 - Appliquer le concept de stride à différentes échelles
- Convolution casuelle (Casual Convolution)
 - C'est essentiellement une convolution 1D
 - Utilisation de paddings pour garder la taille du vecteur



DILATATION

- La dilatation permet d'appliquer les convolutions à des écarts (stride) différents
 - L'idée est d'atteindre des données issues de positions lointaines
 - Réduit aussi le coût computationnel
 - Moins de paramètres dans chaque convolution



EN RÉSUMÉ

- L'utilisation de réseaux de neurones peut donner des très bons résultats avec des séries temporelles
- Les RNN ont un état interne qui fonctionne comme une mémoire
 - Le hidden state est recalculé à chaque nouvelle entrée, avec un risque de perte de gradient
- Les GRU implémentent un "reset gate" qui jauge les mises à jour du hidden state
- Les LSTM passent deux vecteurs à chaque itération :
 - le hidden state (court terme) et le cell-state (long terme)
- D'autres modèles plus récents
- Enfin, **comparer les résultats avec les modèles classiques**, quand possible
 - Le DL peut donner des bons résultats, mais pas toujours
 - Les autres méthodes sont parfois efficaces et beaucoup moins chères